

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE MATEMATICHE, FISICHE E NATURALI



CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA IN

LOGICA E RETI LOGICHE

TITOLO

**Il problema dei Super Hub nella Lightning Network:
simulazioni in Go per l'analisi di attacchi mirati e fallimenti a
cascata**

Relatore:

Prof.
Francesco Pasquale

Candidato:

Nicolò Spadoni
Matricola: 0311175

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Bitcoin e la promessa della decentralizzazione	1
1.2	Il limite della scalabilità e la nascita di Lightning Network	1
1.3	Il paradosso della centralizzazione: l'ascesa dei Super Hub	2
1.4	Panoramica della tesi	2
2	Background tecnologico e teorico	4
2.1	Bitcoin	4
2.1.1	Transazioni, blocchi e mining	4
2.1.2	Chiavi e crittografia a curve ellittiche	5
2.1.3	Script di transazione e firme digitali	6
2.1.4	Indirizzi e firme multisig 2-2	8
2.2	Lightning Network	8
2.2.1	Canali di pagamento	8
2.2.2	Commitment transaction	8
2.2.3	Routing dei pagamenti e contratti HTLC	10
2.2.4	Pathfinding e vincolo della liquidità	12
2.2.5	La distribuzione del grado	13
2.3	Go	14
2.3.1	Libreria Gonum	14
2.3.2	Goroutines	15
3	Il problema	16
3.1	Il processo aleatorio	17
3.2	Obiettivo della tesi	17
4	Architettura del simulatore	19
4.1	Architettura generale	19
4.2	Il package parser	20
4.3	Il package analyzer	21
4.3.1	Costruzione del grafo: il file graph_builder.go	21
4.3.2	Analisi della topologia: il file metriche.go	24
4.4	Il package simulation	25
4.4.1	La scelta dei target: il file attacking.go	25
4.4.2	L'implementazione dei pagamenti: il file payments.go	26
4.4.2.1	Il cuore del package: la funzione calcolaFallimento()	28
5	Analisi dei risultati	30
5.1	La simulazione	30
5.2	La Lightning Network	30
5.2.1	Impatto dei guasti casuali	30
5.2.2	Impatto degli attacchi mirati	32
5.2.3	Aumento del numero di hop	33
5.3	Il modello di Erdős–Rényi–Gilbert	34
5.3.1	La distribuzione dei gradi a confronto	35
5.3.2	Impatto dei guasti casuali	36
5.3.3	Impatto degli attacchi mirati	37
5.3.4	Aumento del numero di hop	38
5.4	Confronto finale	39
5.4.1	Guasti casuali	39

5.4.2 Attacchi mirati	40
5.4.3 Aumento hop	40
6 Sviluppi futuri	41
7 Conclusioni	42
Bibliografia	43

Capitolo 1

Introduzione

1.1 Bitcoin e la promessa della decentralizzazione

Nell'ottobre del 2008, sotto lo pseudonimo di Satoshi Nakamoto, fu pubblicato il white paper “*Bitcoin: a peer-to-peer electronic cash system*” [1]. L'obiettivo era creare un sistema di moneta elettronica *peer-to-peer* in grado di trasferire valore direttamente tra gli utenti, eliminando la necessità di intermediari finanziari centralizzati (come circuiti bancari o enti terzi di pagamento).

Sostituendo la fiducia verso un ente centrale con la crittografia e l'utilizzo di un registro pubblico distribuito noto come *blockchain*, Bitcoin ha dato vita al primo ecosistema finanziario *trustless*, ovvero gli utenti non devono fidarsi né di un autorità centrale, né della controparte che riceve o invia denaro: la validità della transazione è garantita dalle regole matematiche.

Il principio cardine di questa architettura è la *decentralizzazione assoluta*: nessun attore controlla il network, garantendo in questo modo la massima resistenza alla censura e un'estrema sicurezza. Tuttavia, come vedremo, questa impostazione distribuita impone un inevitabile compromesso architetturale.

1.2 Il limite della scalabilità e la nascita di Lightning Network

Nel campo delle reti distribuite — derivato dal *Teorema CAP* [2] — è riconosciuto il concetto di “*Blockchain Trilemma*” [3], secondo cui un'architettura blockchain può ottimizzare solo due di tre proprietà fondamentali: decentralizzazione, sicurezza e scalabilità.

Nella progettazione di Bitcoin, Nakamoto ha scelto di massimizzare le prime due, limitando volontariamente la dimensione dei blocchi e la frequenza della loro emissione. Il limite nella dimensione dei blocchi e l'emissione è necessario per mantenere la natura *peer-to-peer* del sistema, infatti aumentare la dimensione dei blocchi per contenere più transazioni richiederebbe maggiore potenza di calcolo, banda di rete e spazio di archiviazione per la validazione e propagazione dei blocchi. Tutto questo si traduce nella necessità di hardware più costosi, tagliando fuori i nodi di persone comuni e ponendo la rete sotto il controllo di grandi nodi.

Per mantenere la rete sicura e decentralizzata il compromesso è un throughput massimo di circa 7 transazioni al secondo (*TPS*), un valore irrisorio se confrontato con le reti di pagamento tradizionali come Visa, capaci di gestire decine di migliaia di *TPS*.

Per superare questo limite strutturale, nel 2015 è stata introdotta la *Lightning Network* (*LN*) [4]. Si tratta di un protocollo *peer-to-peer* di secondo livello costruito sopra

Bitcoin, che permette agli utenti di scambiarsi fondi *off-chain* tramite una rete di canali di pagamento. Grazie a un sistema di routing interno, LN promette transazioni quasi istantanee e commissioni microscopiche, decongestionando il network sottostante.

1.3 Il paradosso della centralizzazione: l'ascesa dei Super Hub

Essendo Bitcoin una rete decentralizzata, l'aspettativa iniziale era che anche la topologia della Lightning Network si sarebbe come una rete di nodi con grado di connettività simile, in cui ogni utente avrebbe contribuito in modo equo all'instradamento dei pagamenti.

In realtà l'apertura e la gestione di un canale richiedono il pagamento di commissioni, l'immobilizzazione di capitale e il mantenimento di un server sempre connesso. Per la maggior parte degli utenti, l'apertura di più canali di pagamento per risultare ben connesso agli altri, risulterebbe troppo costoso e complesso. La soluzione più efficiente è quella di aprire un singolo canale di pagamento verso un nodo già molto grande e ben connesso come, ad esempio, i nodi gestiti da ACINQ o Kraken.

Questo comportamento ha innescato un'evoluzione topologica fortemente sbilanciata e dominata da pochi nodi centrali denominati *Super Hub*: nodi gestiti da exchange o grandi enti istituzionali che detengono una percentuale sproporzionata di liquidità e di canali aperti. Si assiste così a un paradosso architetturale: un protocollo (LN) progettato per far scalare un ecosistema totalmente decentralizzato (Bitcoin) ha finito per sviluppare forti punti di centralizzazione.

1.4 Panoramica della tesi

L'obiettivo di questa tesi è indagare e misurare la resilienza strutturale e la stabilità della liquidità della Lightning Network di fronte a scenari di potenziale collasso di questi nodi centrali.

Per raggiungere questo scopo, è stato sviluppato un simulatore in linguaggio Go (Golang). Il software analizza sia la topologia della rete, sia lo spostamento dei fondi nei canali, confrontando il comportamento della rete sotto due scenari di stress test: i guasti casuali per esaurimento della liquidità e gli attacchi mirati verso i Super Hub.

La tesi dimostrerà come la caduta dei nodi centrali non si limiti a scollegare gli utenti, ma provochi fallimenti a cascata, a causa della deviazione del traffico su nodi meno centrali che provoca l'esaurimento rapido della liquidità, portando ad una situazione in cui i nodi, seppur fisicamente connessi, sono incapaci di instradare pagamenti.

Per esporre questo studio in modo esaustivo, l'elaborato è stato strutturato in sei capitoli:

Il Capitolo 2 fornisce il background teorico e tecnologico, illustrando il funzionamento di Bitcoin, il funzionamento della Lightning Network (Multisig, HTLC) e presentando gli strumenti informatici scelti per lo sviluppo.

Il Capitolo 3 formalizza il modello matematico alla base della ricerca e definisce il processo aleatorio utilizzato per la simulazione del traffico per analizzare l'impatto della rimozione dei vari nodi.

Il Capitolo 4 presenta l'architettura dell'ambiente di simulazione, analizzando le strutture dati implementate per l'aggiornamento dei saldi in memoria e gli algoritmi utilizzati per la validazione della rete.

Il Capitolo 5 è dedicato all'esecuzione degli stress test e alla discussione dei risultati, quantificando l'impatto topologico ed economico dei Super Hub.

Il Capitolo 6 esplora le possibilità di ampliamento del software sviluppato e le direzioni per sviluppi futuri.

Infine, il Capitolo 7 conclude il lavoro, riassumendo le valutazioni conclusive sulla reale vulnerabilità del Layer 2 di Bitcoin.

Capitolo 2

Background tecnologico e teorico

Questo capitolo fornisce le basi teoriche e tecnologiche necessarie per comprendere l'ambiente di simulazione sviluppato in questo lavoro di tesi. Verrà esaminata l'architettura di base di Bitcoin, il funzionamento della Lightning Network e, infine, gli strumenti informatici scelti per l'implementazione del software.

2.1 Bitcoin

Bitcoin è una criptovaluta decentralizzata basata su un sistema peer-to-peer che opera senza la necessità di un'autorità centrale. In bitcoin la fiducia viene raggiunta dalle interazioni dei diversi partecipanti al sistema. Prima di analizzare la Lightning Network, presentiamo le caratteristiche principali del layer 1, sul quale la rete di secondo livello poggia per garantire l'integrità dei fondi.

2.1.1 Transazioni, blocchi e mining

Una *transazione* comunica alla rete il trasferimento di bitcoin da un utente ad un altro utente, il quale, a sua volta, può spendere i bitcoin creando una nuova transazione verso un nuovo destinatario. A differenza dei sistemi bancari tradizionali, basati su conti correnti, Bitcoin utilizza gli UTXO (Unspent Transaction Output). Una transazione consuma vecchi UTXO per crearne nuovi ed è composta da:

- **Input:** è essenzialmente un puntatore a un output generato in una vecchia transazione ancora non spesa. Per poter sbloccare questi fondi, un utente deve poter fornire una prova crittografica che dimostri il suo diritto di spendere.
- **Output:** definiscono la destinazione dei fondi. Gli output specificano un importo in frazioni di bitcoin — dette *satoshi*— e una condizione crittografica che blocca i fondi verso il nuovo destinatario. Dato che un input deve essere speso nella sua interezza, se il suo valore supera l'importo che vogliamo pagare, allora la transazione genera due output, uno destinato al portafoglio del ricevente e un output di “resto” destinato al portafoglio del mittente.

Una volta definita e firmata digitalmente dall'utente, la transazione non diventa subito effettiva, ma viene propagata in broadcast a tutti gli utenti della rete peer-to-peer. Una volta notate dagli altri nodi, le transazioni vengono aggiunte a una pool temporanea di transazioni non verificate, la *mempool*. Le transazioni in attesa vengono raggruppate in dei *blocchi*, ognuno dei quali mantiene un puntatore hash che si riferisce al blocco precedente, formando una catena immutabile di blocchi: la *blockchain*.

I dati in un blocco dopo essere stati scritti non possono essere retroattivamente alterati, senza che vengano modificati tutti i blocchi successivi, perché questo richiederebbe il consenso della maggioranza della rete.

Tutte le transazioni vengono registrate nella blockchain solamente nel momento in cui il blocco in cui si trovano viene *minato* da nodi della rete detti *miner*. Ciascun nodo può

operare come miner, sfruttando la potenza di calcolo del computer e competendo con gli altri nodi miner alla risoluzione di un complesso problema matematico che, dopo la sua risoluzione, consente la registrazione di un blocco di transazioni nella blockchain. Il nodo miner che ha successo nella risoluzione del problema viene ricompensato con i bitcoin generati dalla registrazione del blocco nella blockchain e le commissioni di tutte le transazioni in esso. Ogni circa 10 minuti un blocco è minato e validato sulla blockchain. La scoperta della soluzione del problema matematico è detta “*Proof of Work*” (PoW) e consiste nell’effettuare l’operazione di hashing dell’header del blocco e un numero casuale con l’algoritmo crittografico SHA256, finché non otteniamo una soluzione corrispondente ad un certo pattern. [5]

2.1.2 Chiavi e crittografia a curve ellittiche

La sicurezza e l’identità degli utenti in Bitcoin si basano sulla *crittografia a chiave asimmetrica* [6]. Sulla rete ogni utente è identificato in modo univoco dalla sua chiave pubblica, che viene derivata da una chiave privata segreta.

Il protocollo Bitcoin utilizza la Crittografia a Curva Ellittica (ECC) [7], basata sullo standard **secp256k1** [8], la cui equazione algebrica è $y^2 \equiv x^3 + 7(\text{mod } p)$ dove p è un numero primo. Essendo questa curva definita su un campo finito di numeri primi invece che su un campo di numeri reali, la sua rappresentazione in due dimensioni è quella di un pattern di punti sparsi, come visibile nella prossima immagine

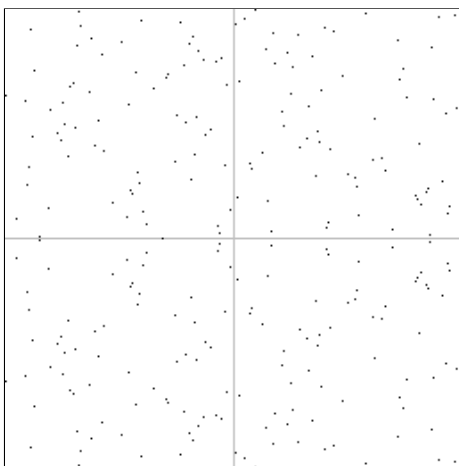


Figura 1: Rappresentazione della curva ellittica su un campo $\mathbb{Z}_p$ di numeri primi

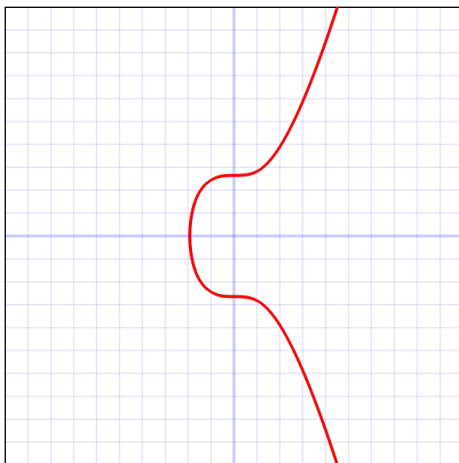


Figura 2: Rappresentazione della curva ellittica su $\mathbb{R}$

In ogni caso la matematica utilizzata per il calcolo della chiave pubblica è identica a quella utilizzata su una curva ellittica definita su un campo finito di numeri primi.

Il calcolo della chiave pubblica inizia con la scelta di una chiave privata, che indichiamo con la lettera k . La chiave privata consiste in un numero a 256 bit scelto in modo casuale. Essa viene utilizzata come parametro da inserire all'interno di un apposito algoritmo — *ECDSA (Elliptic Curve Digital Signature Algorithm)* [9] — per generare le firme da apporre sulle transazioni in uscita che dimostrano la proprietà dei fondi utilizzati, pertanto deve rimanere segreta perché la rivelazione a terzi è equivalente a cedere il controllo dei bitcoin associati a quella chiave.

La chiave pubblica, che indichiamo con la lettera K corrisponde ad un punto di coordinate (x, y) sulla curva ellittica e si calcola tramite moltiplicazione sulla curva ellittica $K = k \cdot G$, dove G è il punto generatore, anch'esso definito nello standard **secp256k1** [8]. L'aspetto principale di questa operazione è la sua *asimmetria* infatti il calcolo in una direzione è semplice, mentre l'operazione inversa, nota come *Problema del Logaritmo Discreto su Curva Ellittica (ECDLP)*, è un problema computazionalmente difficile da risolvere: dato infatti il punto G e un punto $K = k \cdot G$ dovrebbe essere impossibile trovare il valore di k .

La chiave pubblica, oltre ad identificare pubblicamente i nodi nel network, serve anche per la generazione degli indirizzi Bitcoin, l'analogo degli IBAN nei conti correnti bancari. Dato che la chiave pubblica “grezza” è una sequenza di bit molto lunga, per ottimizzarne la dimensione viene sottoposta ad un processo di *hashing* per ottenere un'impronta digitale di dimensione ridotta, secondo un processo noto come *digest*. Partendo quindi dalla chiave pubblica K , si calcola l'hash SHA256 e di questo risultato si calcola l'hash RIPEMD160 producendo un numero a 160 bit che rappresenta l'indirizzo Bitcoin che indichiamo con A , quindi

$$A = \text{RIPEMD160}(\text{SHA256}(K))$$

In questo contesto possiamo sottolineare la robustezza dell'algoritmo di hashing SHA256, che produce sempre un output di 256 bit, quindi il suo spazio totale delle possibili combinazioni è pari a 2^{256} , un numero paragonabile alla stima di atomi nell'universo osservabile, il che rende l'algoritmo a prova di collisioni, perché dal punto di vista statistico e computazionale è impossibile trovare casualmente due preimmagini che producono lo stesso risultato. Questo garantisce quindi che ogni indirizzo generato sia univoco. [10]

2.1.3 Script di transazione e firme digitali

In Bitcoin è presente un linguaggio di scripting, detto *Bitcoin Script* che controlla l'accesso agli output delle transazioni. Finora ci siamo riferiti alle transazioni con la semplice forma di “Alice paga Bob”, ma in realtà, grazie a *Bitcoin Script* possiamo implementare condizioni di spesa più complesse. La più comune e utilizzata è una condizione di spesa che possiamo definire come “presentare una firma che corrisponda a questa chiave pubblica”. Bitcoin Script è un linguaggio di scripting basato su stack molto semplice, non contiene loop o ricorsioni rendendolo quindi Turing incompleto, garantendo una complessità limitata ed esecuzioni prevedibili, oltre ad aumentare la sicurezza in caso di utilizzi malevoli. [5]

Per validare le transazioni, viene utilizzato Bitcoin Script per la costruzione di due tipi di script:

- **script di blocco:** viene posto sugli output delle transazioni e specifica le condizioni necessarie da soddisfare per poter spendere quell'UTXO, come ad esempio la presentazione di una firma valida del destinatario per la spesa.
- **script di sblocco:** sono posti sugli input delle transazioni per risolvere le condizioni poste in precedenza da uno script di blocco su un output, consentendo di consumare l'UTXO.

Ogni client bitcoin convaliderà le transazioni eseguendo insieme gli script di blocco e sblocco. Per ogni input della transazione, vengono prima recuperati gli UTXO utilizzati, ciascuno dei quali conterrà uno script di blocco. Viene recuperato in seguito lo script di sblocco contenuto nell'input, concatenato allo script di blocco ed eseguiti in sequenza, validando l'input se l'esecuzione termina con il risultato **TRUE**

La maggioranza delle transazioni sulla rete Bitcoin sono transazioni *P2PKH*, il cui script di blocco contiene l'hash della chiave pubblica del destinatario. Le transazioni P2PKH possono essere sbloccate (e quindi spese) presentando la chiave pubblica e una firma digitale creata dalla corrispondente chiave privata. Nel caso di una transazione da Alice a Bob l'output della transazione avrebbe uno script di blocco simile al seguente:

```
OP_DUP OP_HASH160 <Bob Public Key Hash> OP_EQUALVERIFY OP_CHECKSIG
```

che può essere soddisfatto dallo script di sblocco seguente:

```
<Bob Signature> <Bob Public Key>
```

e combinati insieme vanno a costituire lo script di validazione

```
<Bob Signature> <Bob Public Key> OP_DUP OP_HASH160 <Bob Public Key Hash>  
OP_EQUALVERIFY OP_CHECKSIG
```

L'esecuzione dello script di validazione restituirà **TRUE** se e solo se lo script di sblocco presenta una firma valida dalla chiave privata di Bob, che corrisponde all'hash della chiave pubblica.

In Bitcoin, per comprovare la proprietà di una chiave privata, senza rivelarla direttamente, vengono utilizzate le *firme digitali*. Questa verifica viene riassunta in due fasi: una prima fase che consiste nell'esecuzione di un algoritmo (ECDSA) per la creazione della firma utilizzando la chiave privata e un messaggio (la transazione); la seconda fase è un algoritmo che consente la verifica della firma, dato il messaggio e la chiave pubblica.

Per la creazione di una firma digitale, come messaggio viene utilizzato un hash di un sottoinsieme di dati della transazioni e la chiave privata dell'utente, ottenendo:

$$S = F_{\text{sig}}(F_{\text{hash}}(m), k)$$

dove k è la chiave privata, m è il messaggio, F_{hash} è la funzione di hashing, F_{sig} è l'algoritmo di firma ed S è la firma risultante.

Per la verifica occorre avere la firma, l'hash della transazione e la chiave pubblica e l'algoritmo restituirà **TRUE** se la firma è valida per il messaggio e la chiave pubblica.

2.1.4 Indirizzi e firme multisig 2-2

Il linguaggio di scripting consente di creare condizioni di spesa più complesse rispetto la firma singola vista in precedenza: una di queste è la *multisig*. In un indirizzo multisig, la condizione di blocco viene impostata per richiedere M firme valide su un totale di N chiavi pubbliche fornite, note come schema M-di-N. Nella Lightning Network, la configurazione fondamentale è quella del Multisig 2-di-2, in cui i fondi vengono bloccati in un indirizzo multisig appartenente a due utenti differenti e affinché la transazione in input sia spendibile è necessario fornire entrambe le firme digitali dei due utenti.

2.2 Lightning Network

Come visto nel capitolo di introduzione, l'architettura blockchain di Bitcoin massimizza sicurezza e decentralizzazione, ma ha dei limiti nella scalabilità, consentendo l'esecuzione di circa 7 transazioni al secondo. Il protocollo Lightning Network nasce con l'obiettivo di risolvere proprio questo problema, riducendo i costi e i tempi richiesti per le transazioni *on-chain*, mantenendo le garanzie di sicurezza di Bitcoin.

2.2.1 Canali di pagamento

La base fondamentale della Lightning Network è il *canale di pagamento bidirezionale*. Questo canale è gestito da un protocollo crittografico, utilizzato dai due partner di canale per ridistribuire il saldo del canale, garantendo che nessuno dei due partner di canale possa imbrogliare l'altro. Il protocollo si basa sull'invio di fondi ad un indirizzo multisig 2-di-2 tramite una transazione iniziale detta *Funding Transaction* che blocca dei fondi che costituiscono la *capacità* del canale di pagamento. L'utilizzo di questo indirizzo richiede la collaborazione dei due partner ed impedisce di spendere i fondi unilateralmente.

La Funding Transaction permette quindi l'apertura di un canale lightning ed è una transazione che viene effettuata *on-chain*, quindi necessita di conferma sulla blockchain dai miner. Le transazioni di apertura del canale e di chiusura (*Closing Transaction*) sono le uniche transazioni che avvengono *on-chain*. Al contrario tutti gli aggiornamenti di bilancio tra i nodi della rete Lightning avvengono *off-chain* tramite lo scambio di transazioni dette *commitment transactions*. Ogni volta che viene creata una nuova commitment transaction, viene invalidata e sostituita la precedente, per questo motivo i pagamenti sulla Lightning Network sono più veloci, perché non attendono i tempi di conferma della Proof of Work per la validazione sulla blockchain.

2.2.2 Commitment transaction

Per aggiornare il bilancio tra i due nodi del canale, i due partner si scambiano le commitment transactions che riflettono la divisione attuale dei bitcoin nel canale tra i due nodi. Quando i due nodi effettuano uno scambio di bitcoin sul canale di pagamento, generano e firmano le due commitment transactions, concordando un nuovo bilancio del canale. Le due transazioni di impegno sono asimmetriche, e ognuno mantiene una copia speculare della transazione che indica quanti *satoshi* sono dalla propria parte del canale e quanti dalla parte del partner.

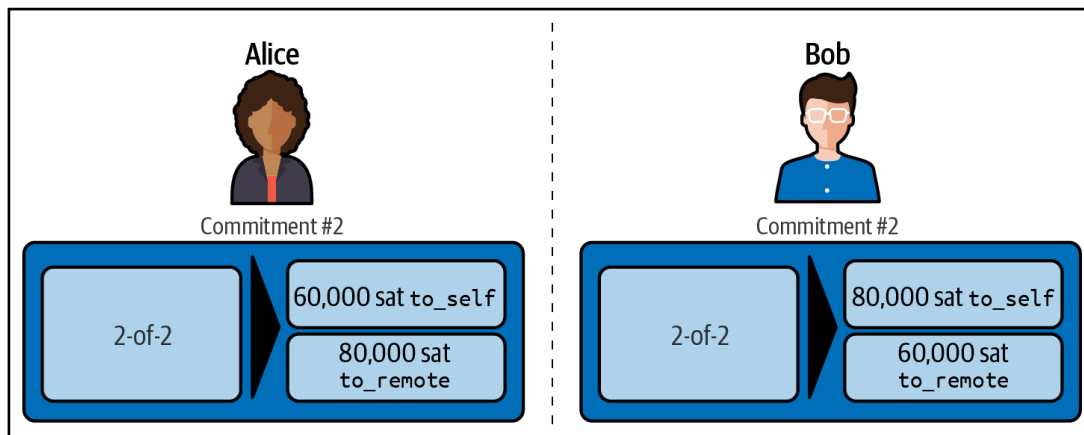


Figura 3: Rappresentazione delle due transazioni di impegno asimmetriche

Per aggiungere un livello in più di sicurezza e “disincentivare” gli utenti a barare, viene utilizzato un meccanismo detto *timelocked*, ovvero, prima che un utente possa spendere i suoi satoshi, deve attendere un numero prestabilito di tempo, misurato in blocchi minati, come visibile nella prossima figura

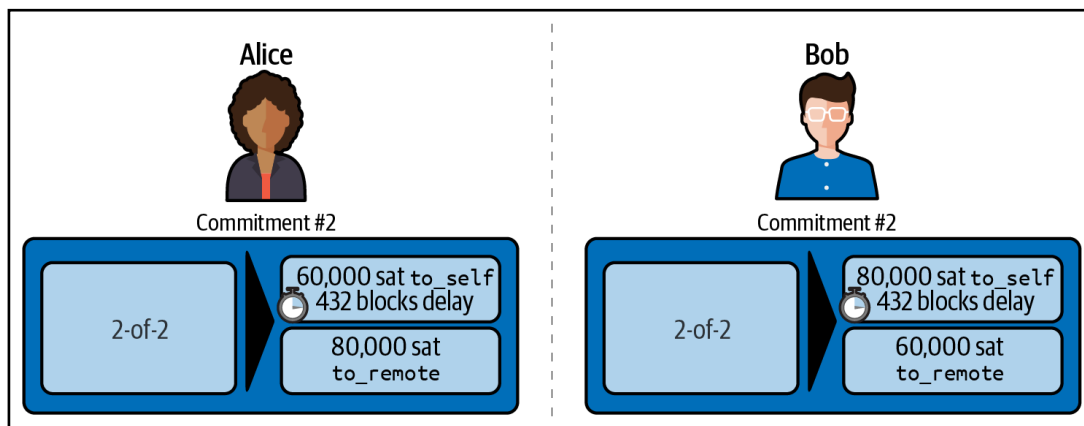


Figura 4: Rappresentazione delle transazioni di impegno timelocked

in questo caso, sia Alice che Bob possono spendere i satoshi destinati a loro dopo il mining del blocco numero 432. Questo ritardo consente ad uno dei due nodi di esercitare un'operazione di penalità, nel caso in cui l'altro partner di canale dovesse trasmettere una transazione di impegno che non rispecchia l'ultimo aggiornamento di bilancio.

L'ultima procedura di sicurezza che viene inserita è l'utilizzo di una *chiave di revoca*. Questo permette di avanzare nello stato del bilancio del canale: dopo aver creato e scambiato le firme per la transazione di impegno che porta nello stato $n + 1$, vengono scambiate le chiavi di revoca relative allo stato precedente n . Vedendo la transazione di impegno dalla parte di Alice, questa ora si evolve in uno script di blocco a due condizioni:

- i fondi possono essere spesi da Alice dopo la scadenza del timelock
- i fondi possono essere spesi immediatamente da Bob se è in possesso della chiave di revoca dello stato precedente

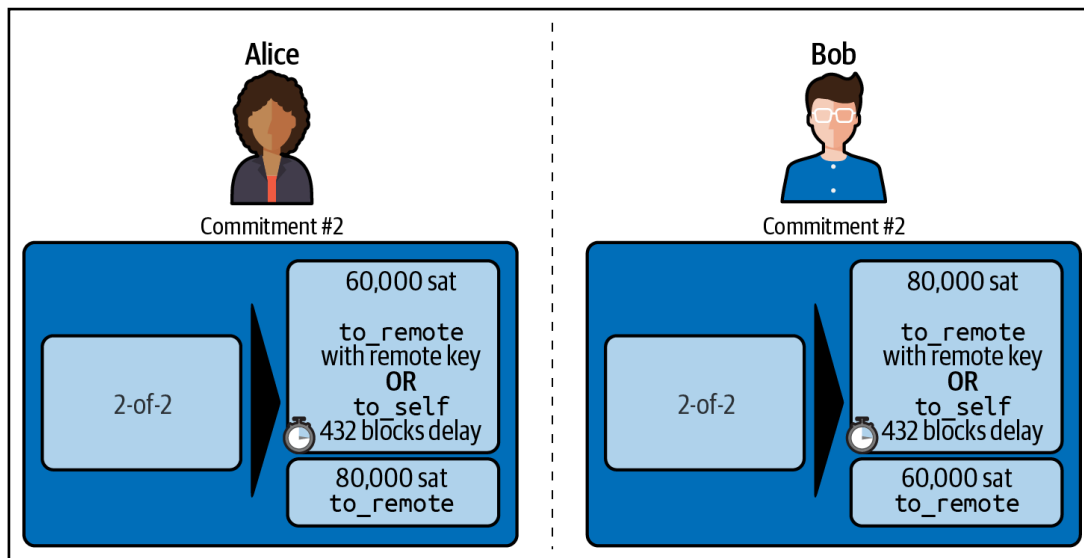


Figura 5: Rappresentazione delle transazioni di impegno timelocked

Nel caso in cui Alice, tentando di truffare Bob pubblici sulla blockchain una transazione di impegno precedente all'ultimo stato — che indicheremo con stato n —, deve attendere il mining di 432 blocchi (circa 3 giorni) per poter spendere quei bitcoin. Durante questo tempo, Bob, monitorando la blockchain, può individuare la transazione non autorizzata, ed essendosi scambiati la chiave di revoca nell'aggiornamento di stato $n + 1$, può creare una transazione di penalità, incassando sia i propri fondi, sia quelli destinati ad Alice.

L'utilizzo di questi meccanismi garantisce la sicurezza del protocollo di secondo livello, infatti qualsiasi tentativo di furto non solo viene reso fallimentare, ma causa anche la perdita totale di tutti i fondi da parte di chi lo commette, costringendo quindi i partecipanti a rimanere onesti.

La procedura di chiusura del canale avviene con la pubblicazione dell'ultima transazione di impegno che corrisponde all'ultimo stato del bilancio del canale ed è a tutti gli effetti una transazione *on-chain* che viene registrata sulla blockchain. La chiusura può avvenire in due modi distinti:

- **chiusura cooperativa:** rappresenta lo scenario di chiusura ideale. Quando Alice e Bob decidono di chiudere il proprio canale, firmano la transazione di chiusura che distribuisce i fondi concordati nell'ultima commitment transaction. Essendo un'operazione concordata da entrambi i nodi non viene imposto nessun timelock, quindi i fondi di entrambe le parti sono immediatamente spendibili
- **chiusura unilaterale:** la chiusura unilaterale, oltre che nel caso di tentativo di frode, è necessaria nel caso in cui uno dei due partner di canale si rifiuta di collaborare o diventa irraggiungibile a causa di guasti. In caso di assenza di risposte da parte di Bob, Alice può quindi pubblicare in autonomia l'ultima transazione di impegno valida sulla blockchain. Poiché l'azione è unilaterale, i fondi di Alice sono bloccati dal timelock, permettendo comunque di avere una finestra temporale per eventuali contestazioni.

2.2.3 Routing dei pagamenti e contratti HTLC

Cosa succederebbe nel caso in cui nella nostra rete entrasse Charlie, un nuovo utente? Se ora Alice volesse effettuare un pagamento a Charlie potrebbe aprire un canale

diretto con lui. Chiaramente se espandiamo questo approccio a tutti gli utenti mondiali che vorrebbero utilizzare la Lightning Network torneremo di nuovo sul problema della scalabilità di Bitcoin!

Ipotizziamo quindi che in realtà Alice voglia pagare Charlie e quest'ultimo ha un canale diretto con Bob (che a sua volta ne ha uno con Alice). Il potenziale della Lightning Network esce fuori proprio in questo caso, infatti Alice può inviare fondi a Charlie facendoli passare attraverso il nodo di Bob. C'è quindi bisogno di un meccanismo per impedire a Bob di rubare i fondi che deve instradare verso Charlie, tramite un protocollo basato su degli *smart contract* chiamati *Hashed Timelocked Contracts (HTLC)*.

Gli HTLC sono dei contratti crittografici che utilizzano l'hash di una *preimmagine di pagamento segreta*, che consiste in un numero casuale generato dal ricevente dei fondi. Denotiamo questo segreto con R e calcoliamo il suo hash crittografico per ottenere l'*hash di pagamento*, che indichiamo con H dove

$$H = \text{SHA256}(R)$$

Gli HTLC contengono dunque due elementi:

- un **hashlock**: ovvero un lucchetto che blocca i fondi. Questi possono essere spesi solamente se viene fornita la preimmagine che produce l'hash di pagamento.
- un **timelock**: come visto in precedenza consiste in un blocco temporale. Se il contratto non viene sbloccato entro la scadenza del tempo, i fondi tornano automaticamente al mittente.

Supponiamo quindi che Alice voglia pagare Charlie passando per il nodo intermediario di Bob e vediamo nel dettaglio il procedimento che permette il pagamento.

La prima fase consiste nella generazione della preimmagine segreta da parte di Charlie, ovvero un numero casuale R , di cui calcola l'hash $H = \text{SHA256}(R)$. Questo hash viene trasmesso ad Alice, quindi Charlie è l'unico a conoscenza della preimmagine che genera l'hash H

Alice a questo punto crea con Bob un HTLC che afferma: “Bob, trasferirò questi fondi (più una commissione) verso la tua parte del canale se mi fornirai il segreto R che produce l'hash H entro il blocco 1000 (timelock T_1). Se Bob non fornisce il segreto R entro questo tempo, i fondi depositati verranno rimborsati e il contratto non sarà più valido”.

Bob, però, non conosce il segreto R , ma sa che Charlie possiede questo segreto, quindi crea un HTLC nel suo canale con Charlie dicendogli: “Charlie, trasferirò questi fondi verso il tuo canale se mi fornirai il segreto R che produce l'hash H entro il blocco 900 (timelock T_2). Se questo segreto non viene fornito entro la scadenza del tempo, i fondi depositati verranno rimborsati e il contratto non sarà più valido”.

Specifichiamo che in questa fase ogni tratta del percorso richiede la creazione di un HTLC indipendente, ovvero uno esclusivo tra Alice e Bob e uno esclusivo tra Bob e Charlie, destinato ai rispettivi indirizzi multisig 2-di-2. In caso contrario, il destinatario del pagamento, essendo l'unico a conoscere il segreto potrebbe riscattare tutti gli HTLC del percorso, mentre in questo caso Charlie può riscattare solamente i fondi all'interno del canale tra lui e Bob.

Osserviamo inoltre che $T_2 < T_1$: questa asimmetria temporale è cruciale, perché consente a Bob di avere il tempo necessario di ottenere i fondi di Alice subito dopo aver pagato Charlie.

L'ultima fase consiste nella risoluzione a catena del pagamento: Charlie rivela a Bob il segreto R risolvendo l'hashlock del loro HTLC e incassando i fondi. Ora Bob è a conoscenza del segreto R e mostrandolo ad Alice può anch'esso incassare i fondi (e la commissione per aver fatto da intermediario nel pagamento) rimborsandosi di quanto anticipato a Charlie.

L'HTLC garantisce l'*atomicità* nel pagamento: o la transazione va a buon fine per tutti i partecipanti sbloccandosi a catena dall'ultimo al primo o fallisce per tutti. Se Charlie diventasse improvvisamente irraggiungibile prima di rivelare il segreto R allora allo scadere del blocco 900 l'HTLC tra Bob e Charlie si annullerebbe e di conseguenza al blocco 1000 scadrebbe anche l'HTLC tra Alice e Bob. In questo caso il pagamento fallirebbe, ma nessuno perderebbe i propri fondi.

2.2.4 Pathfinding e vincolo della liquidità

Nella Lightning Network è il mittente che si occupa della ricerca di un percorso su cui instradare il pagamento. Questa procedura è chiamata *pathfinding* ed unisce elementi del *source routing* — in cui il percorso viene selezionato e specificato dal mittente — e dell'*onion routing* [11].

L'onion routing è utilizzato per garantire un livello di privacy in più, infatti, diversamente da quanto avviene nel routing Internet, in cui ogni nodo intermedio è a conoscenza di mittente e destinatario, l'approccio utilizzato è quello di crittografare il messaggio del mittente in più livelli nidificati (come gli strati di una cipolla) così che i nodi intermedi abbiano informazioni solamente sul nodo da cui ricevono il messaggio e il nodo a cui devono consegnarlo, senza avere informazioni sull'effettivo destinatario del messaggio o sulla lunghezza del percorso. Quest'ultimo aspetto è reso possibile dall'utilizzo di “cipolle” a lunghezza fissa, ovvero dei messaggi a cui, volta decrittografati, vengono aggiunti dei “dati spazzatura” alla fine del payload, in modo tale che ad ogni hop il nodo intermedio veda la dimensione del messaggio come se fosse all'inizio del suo percorso di instradamento. Lightning Network utilizza un'implementazione del protocollo di onion routing basato su SPHINX [12].

Il pathfinding deve risolvere un problema di ricerca del percorso “migliore” tra due nodi per instradare il pagamento. La scelta quindi deve bilanciare diversi fattori, ovvero si vuole minimizzare il totale delle commissioni richieste dai nodi intermedi, massimizzare la probabilità di instradamento, minimizzare i timelock per evitare che i fondi restino bloccati per troppo tempo e infine scegliere i percorsi in cui ogni canale abbia liquidità sufficiente per far passare il pagamento.

La rete della Lightning Network può essere rappresentata mediante un grafo pesato $G = (V, E, c)$ in cui l'insieme V è l'insieme dei nodi, ovvero gli utenti che hanno un portafoglio lightning, l'insieme degli archi E rappresenta l'insieme di tutti i canali di pagamento tra due nodi, ovvero $E = \{u, v \in V : u \text{ ha un canale di pagamento con } v\}$ e $c : E \rightarrow \mathbb{N}$ è una funzione di costo che indica la capacità di un canale $\{u, v\}$. Il limite principale del pathfinding è che per motivi di privacy i nodi sono a conoscenza solamente dei bilanci sui canali che hanno direttamente aperto e non conoscono i saldi degli altri nodi della rete, dunque l'algoritmo di routing potrebbe scegliere un percorso ottimale

ma che, per mancanza di liquidità in uno dei nodi intermedi, porterebbe al fallimento del pagamento.

A causa di questo ultimo vincolo, la ricerca del percorso di instradamento segue una logica del tipo *trial and error* in cui, mediante un algoritmo di ricerca di un cammino da mittente a destinatario, viene creato un elenco di percorsi candidati, ordinati secondo una caratteristica da ottimizzare (commissioni, timelock, probabilità di successo) e viene effettuato un ciclo di tentativi ed errori finché non viene trovato un percorso che consegna con successo il pagamento. Nel caso in cui durante un tentativo di pagamento venga trovata una rotta con liquidità insufficiente, questo fallisce e il nodo mittente aggiorna il suo “grafo dei canali”, ovvero la mappa di tutti i nodi che può raggiungere costruita mediante il *gossip protocol* [13], escludendo temporaneamente il canale che ha fatto fallire il pagamento.

2.2.5 La distribuzione del grado

La struttura topologica della Lightning Network ha contribuito a distribuire il grado dei nodi in modo non uniforme. Nelle prossime due figure possiamo osservare come siano presenti molti nodi con grado basso — ovvero con pochi canali aperti — e pochissimi nodi con una grande quantità di canali, che durante la trattazione indicheremo come *super hub*.

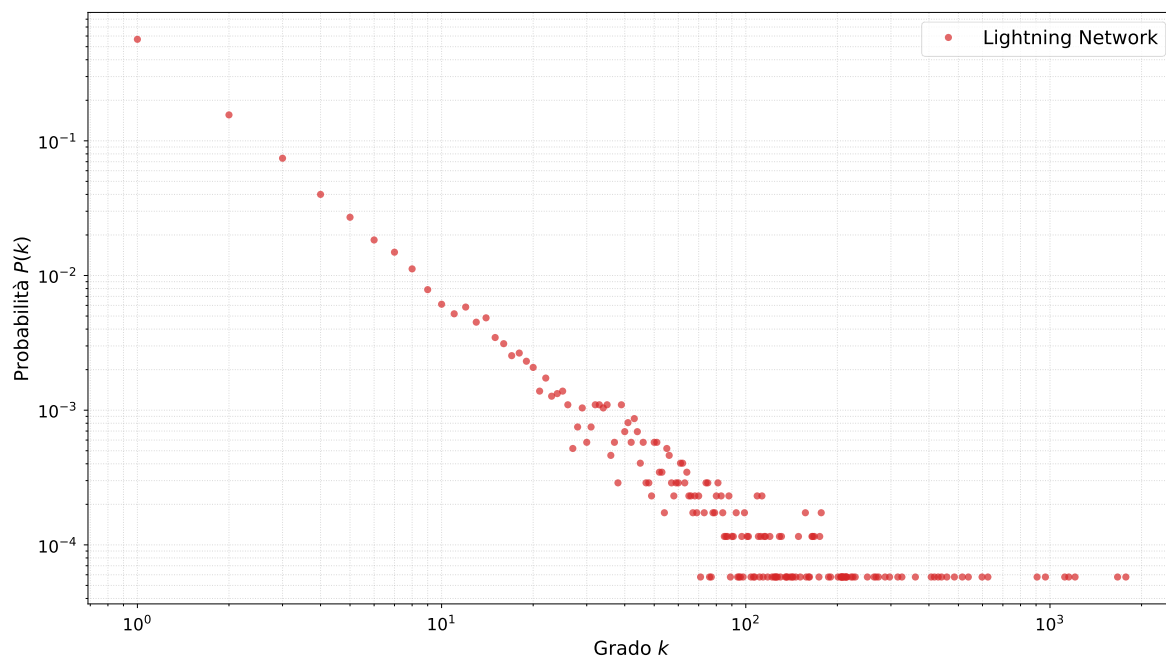


Figura 6: Distribuzione del grado della LN

Osserviamo che, all’aumentare del grado, la probabilità $P(k)$ di pescare casualmente un nodo di grado k diminuisce drasticamente. La distribuzione segue una legge di potenza, tipica delle reti *scale-free* [14]. Questo dimostra l’esistenza dei super hub in LN, ovvero quei nodi che, seppur rari da trovare, hanno un grado $\deg(v) > 10^3 = 1000$, su cui l’intera rete poggia.

Il grafo della Lightning Network globale può essere rappresentato nella seguente figura, in uno screenshot tratto dal sito mempool.space, che mostra la rete al 07/07/2026.



Figura 7: Rappresentazione del grafo globale della Lightning Network

2.3 Go

Go, sviluppato nel 2009 da Robert Griesemer, Rob Pike e Ken Thompson di Google, è un linguaggio di programmazione compilato e fortemente tipizzato che unisce l'efficienza e le prestazioni del C ad una sintassi più semplice e moderna. Per analizzare la topologia e simulare i pagamenti, questo linguaggio si è rivelato molto utile per lo sviluppo di un simulatore ad alte prestazioni.

Il software acquisisce uno *snapshot* della Lightning Network. Nonostante la struttura del grafo di input (nodi e canali) rimanga immutata durante l'esecuzione, lo stato interno dei canali, ovvero la liquidità, varia in ogni momento durante la simulazione dei pagamenti. Per questo motivo, la gestione di migliaia di nodi e l'aggiornamento dei vari parametri consultati durante la simulazione, richiede una gestione efficiente della memoria.

La scelta di Go ricade quindi sulla necessità di avere un programma che possa gestire in modo veloce ed efficiente l'enorme quantità di dati in input. La gestione della memoria è affidata ad un *garbage collector*, ma a differenza di linguaggi come Python, permette la manipolazione diretta dei dati in memoria mediante l'utilizzo dei puntatori.

Nella simulazione dei pagamenti, i canali sono rappresentati da specifiche **struct** che, una volta caricato tutto il grafo della Lightning Network, è possibile aggiornarne i dati in tempo costante $O(1)$ modificando direttamente i valori all'indirizzo di memoria del canale.

2.3.1 Libreria Gonum

Per la gestione efficiente del grafo la scelta è ricaduta sull'insieme di librerie **Gonum**, nello specifico sul pacchetto **gonum/graph** che fornisce gli strumenti per la costruzione e l'analisi di reti complesse.

Tramite la libreria, la rete Lightning viene mappata in un grafo non orientato, i cui archi rappresentano gli indirizzi multisig 2-di-2. Gonum si è rivelata molto utile per il

calcolo di diverse metriche nei grafi, grazie alla sua alta disponibilità di algoritmi nativi efficienti, quali:

- il calcolo delle componenti connesse, utilizzato per l'estrazione della *Giant Component*, ovvero la componente connessa di dimensione maggiore, che esclude tutti i nodi isolati
- il calcolo della degree centrality, del grado e delle metriche utili ad individuare i *super hub* e simulare gli attacchi mirati

Nonostante Gonom offra degli algoritmi per il calcolo degli shortest path, come Dijkstra o Bellman-Ford, la natura dinamica della Lightning Network non permetteva l'utilizzo diretto di questi. Il vincolo della liquidità richiede che un arco, per essere attraversato, debba possedere fondi sufficienti nella direzione richiesta. Questo ha quindi portato alla necessità di implementare una visita BFS personalizzata che, esplorando i canali del grafo, scarta quelli con liquidità insufficiente.

2.3.2 Goroutines

Go è progettato per la creazione di un software scalabile e veloce. Se dobbiamo gestire un grande numero di dati, i tempi di attesa per le varie operazioni potrebbero rallentare l'esecuzione. Per risolvere questa problematica in Go possiamo utilizzare le **goroutine**, dei thread molto leggeri e gestiti dalla runtime di Go, invece che dal sistema operativo come avviene in altri linguaggi di programmazione.

Le **goroutine** si sono rivelate utili per parallelizzare l'esecuzione dei test su 4 scenari diversi: i guasti casuali e gli attacchi mirati sul grafo della Lightning Network e gli stessi scenari su un grafo random generato a partire dal modello di Erdős–Rényi.

Capitolo 3

Il problema

Prendendo spunto dal lavoro svolto in [15] modelliamo matematicamente il problema che analizzeremo in uno dei prossimi capitoli.

La Lightning Network può essere rappresentata come un grafo non orientato e pesato $G = (V, E, c)$, dove V rappresenta l'insieme dei nodi del protocollo mentre E rappresenta l'insieme dei canali di pagamento esistenti tra coppie di nodi.

La funzione $c : E \rightarrow \mathbb{N}$ è la *funzione di capacità* che associa ad ogni arco e un numero naturale $c(e)$ che rappresenta la capacità massima del canale, condivisa tra i due vertici dell'arco per costituire il bilancio (privato) dei due partner di canale. Possiamo rappresentare i bilanci come una coppia di valori $\{b_e(u), b_e(v)\}$ tale che $b_e(u), b_e(v) \in \{0, 1, \dots, c(e)\}$ e $b_e(u) + b_e(v) = c(e)$.

Un pagamento diretto tra due nodi adiacenti non è altro che l'aggiornamento del bilancio del canale $e = \{u, v\}$, ovvero se il bilancio attuale è $\{b_e(u), b_e(v)\}$ ed u esegue un pagamento di una quantità di $x \in \{1, \dots, b_e(u)\}$ satoshi verso v , allora il nuovo bilancio dopo la transazione diventa $\{b_e(u) - x, b_e(v) + x\}$.

I pagamenti possono essere instradati attraverso tutto il grafo, connettendo nodi non adiacenti. Se un nodo u volesse inviare un pagamento ad un nodo destinatario v non adiacente, deve ricercare un path $\mathcal{P} = (u = u_0, u_1, \dots, u_h = v)$ nel grafo. Affinché il pagamento di una quantità x di satoshi vada a buon fine, la liquidità in ciascun canale deve essere sufficiente, quindi, per ogni arco $e_i = \{u_i, u_{i+1}\}$ deve valere $x \leq b_{e_i}(u_i)$. La buona riuscita del pagamento aggiornerà i bilanci di ogni canale $e_i = \{u_i, u_{i+1}\}$ nel path $\mathcal{P}$, il cui nuovo bilancio $\{b'_{e_i}(u_i), b'_{e_i}(u_{i+1})\}$ diventerà:

$$\begin{cases} b'_{e_i}(u_i) = b_{e_i}(u_i) - x \\ b'_{e_i}(u_{i+1}) = b_{e_i}(u_{i+1}) + x \end{cases}$$

Mentre la capacità di ogni canale è un'informazione pubblica propagata tramite il *protocollo gossip* [13], i bilanci sono privati e conosciuti solamente dai partner di canale. A causa di questo vincolo è possibile che durante il tentativo di pagamento lungo un percorso $\mathcal{P}$, si incontri un canale $e_i = \{u_i, u_{i+1}\}$ il cui bilancio non è sufficiente per eseguire il pagamento, ovvero $x > b_{e_i}(u_i)$. In questo caso il pagamento fallisce ed è necessaria la ricerca di un nuovo path $\mathcal{P}'$ per eseguire il pagamento.

3.1 Il processo aleatorio

Per l'analisi del problema dell'importanza dei super hub nella rete Lightning definiamo il seguente processo aleatorio.

Consideriamo un grafo $G = (V, E, c)$ in cui assumiamo che il bilancio di ogni arco $e = \{u, v\} \in E$ sia equamente diviso rispetto alla sua capacità totale $c(e)$, quindi $b_e(u) = b_e(v) = \frac{c(e)}{2}$.

La simulazione si svolge in una sequenza di round $t = 1, 2, \dots, N$. Ad ogni round t scegliamo uniformemente a caso (u.a.r.) due nodi $u, v \in V$ tali che $u \neq v$, rispettivamente il nodo mittente e destinatario. Simuliamo il pagamento di una quantità fissa di x satoshi cercando un path $\mathcal{P} = (u = u_0, u_1, \dots, u_h = v)$ tra u e v tale che $\forall i \in \{0, \dots, h\} b_{e_i}(u_i) \geq x$, utilizzando una ricerca BFS [16], con complessità temporale $O(|V| + |E|)$, modificata per accedere ai bilanci e scartare gli archi con una liquidità inferiore al pagamento x .

Pseudocodice 1 Il processo aleatorio

Richiede: grafo $G = (V, E)$; numero pagamenti N , importo pagamento x

```

1  successi  $\leftarrow$  0; failLiq  $\leftarrow$  0; failPath  $\leftarrow$  0; totHop  $\leftarrow$  0
2  for  $i = 0$  to  $N$  do
3      RIPRISTINABILANCIO ( $G$ )
4      sorg  $\leftarrow$  scegli nodo sorgente  $u$  u.a.r. da  $V$ 
5      dest  $\leftarrow$  scegli nodo destinazione  $v$  u.a.r. da  $V$ 
6      esito, hops, pathEsiste  $\leftarrow$  PAGA (sorg, dest,  $x$ ,  $G$ )
7      if esito == true
8          successi++
9          totHop += hops
10     else if pathEsiste == true
11         failLiq++
12     else
13         failPath++
14     end for
15     tassoSuccesso  $\leftarrow$  successi/ $N$ 
16     tassoFailLiq  $\leftarrow$  failLiq/( $N$  - failPath)
17     tassoFailPath  $\leftarrow$  failPath/ $N$ 
18     if successi > 0
19         avgHops  $\leftarrow$  totHop/successi
20     return tassoFailLiq, tassoFailPath, avgHops

```

3.2 Obiettivo della tesi

Il pagamento può avere tre esiti:

- **successo:** viene trovato un percorso $\mathcal{P}$ i cui archi hanno liquidità sufficiente per inoltrare il pagamento. I bilanci vengono aggiornati.

- **fallimento per liquidità:** nonostante esista un percorso tra u e v , nessuno di questi ha liquidità sufficiente in un arco per inoltrare il pagamento. Il tasso di questo fallimento deve essere calcolato non sul totale dei tentativi, bensì su tutti quei tentativi di pagamento per cui un path esiste. Per questo motivo il denominatore è $N - \text{failPath}$, dove N è il numero totale di tentativi e failPath è il contatore che indica quanti pagamenti sono falliti per mancanza di un path tra u e v .
- **fallimento per assenza di path:** questo fallimento avviene quando non esiste un path tra i due nodi u e v .

L'obiettivo della tesi è quello di misurare come varia il tasso di successo in relazione al fallimento fisico dei nodi della rete. Per ottenere questo risultato il processo aleatorio viene ripetuto più volte, andando a configurare diversi scenari nel grafo in input:

- **grafo intatto:** il processo è eseguito sulla topologia pura del network.
- **guasti casuali:** dal grafo vengono scelti u.a.r. e rimossi uno per volta un numero di k nodi, simulando guasti hardware o perdita di connessione non intenzionale. Ad ogni rimozione viene eseguito il processo di N round.
- **attacchi mirati:** vengono rimossi progressivamente dal grafo i primi k nodi ordinati in modo decrescente rispetto la loro *degree centrality*, simulando un attacco contro i *super hub*. Anche in questo caso ad ogni rimozione viene eseguito il processo di N round.

Prima di ogni esecuzione del processo di N round, lo stato dei bilanci della rete viene ripristinata alla condizione iniziale di $\frac{c(e)}{2}$. Questo serve a garantire che i fallimenti siano dovuti all'effettivo attacco topologico e non all'esaurimento dei fondi delle esecuzioni precedenti.

Le simulazioni vengono effettuate su due grafi distinti: la giant component estratta dal grafo della Lightning Network e la giant component estratta dal grafo $G(n, p)$ generato a partire dal modello di Erdős–Rényi con $p = 0.000336$, per fare in modo che il numero di archi di $G(n, p)$ sia simile al numero di archi della Lightning Network.

Il confronto tra i tassi di fallimento dei vari scenari ci fornirà la dimostrazione della centralizzazione della Lightning Network, mostrando l'impatto dei super hub nell'efficienza dell'instradamento nella rete.

Capitolo 4

Architettura del simulatore

Dopo aver presentato gli aspetti teorici dietro Bitcoin e Lightning Network, questo capitolo ha l'obiettivo di presentare l'architettura del software sviluppato in Go, analizzando le scelte implementative e l'ottimizzazione delle strutture dati in memoria per la simulazione dei pagamenti su classi di grafi differenti.

4.1 Architettura generale

Il programma è stato progettato seguendo i principi del *Package-Oriented Design* del linguaggio Go, separando le funzionalità in moduli indipendenti per garantire la modularità e la manutenibilità del codice. L'applicazione è suddivisa in tre package controllati dal modulo principale `main.go`.

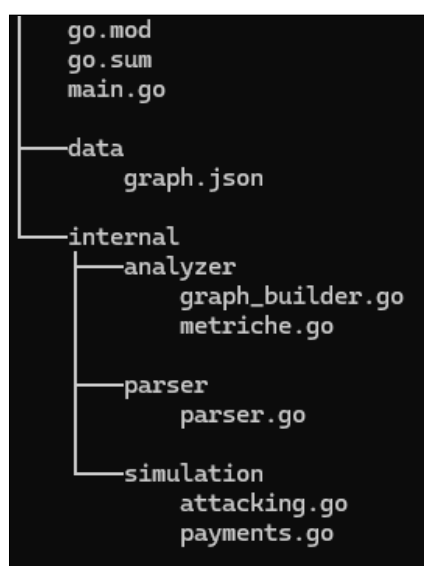


Figura 8: Struttura del programma in Go

I tre package sono i seguenti:

- **parser**: è il modulo dedicato alla decodifica dei dati del file JSON `graph.json`, contenente lo snapshot della rete Lightning al 26/02/2026.
- **analyzer**: è il modulo che implementa la costruzione in memoria del grafo della Lightning Network e il grafo $G(n, p)$ tramite **Gonum**, la creazione delle **struct** per la gestione dei canali di pagamento e per il calcolo delle metriche quali il numero di componenti connesse, il loro diametro e il grado medio di ogni nodo.
- **simulation**: è il modulo operativo in cui viene implementato l'algoritmo di pagamento, vengono individuati i bersagli da attaccare tramite il calcolo della degree centrality ed effettua le simulazioni sui diversi grafi dati in input.

4.2 Il package parser

Il primo passo per iniziare la simulazione è l'estrazione dei dati dallo snapshot della rete, rappresentata nel file di grande dimensione **graph.json** che contiene l'elenco di tutti i nodi e di tutti i canali aperti nella rete Lightning.

Dato che il file JSON contiene diversi metadati associati sia ai nodi che agli archi, definiamo due **struct** con lo scopo di immagazzinare le uniche informazioni necessarie. Attraverso i tag JSON indichiamo al parser come mappare i dati del JSON all'interno della struct

```
type NodeData struct {
    PubKey string `json:"pub_key"`
    Alias  string `json:"alias"`
}

type EdgeData struct {
    Node1pub string `json:"node1_pub"`
    Node2pub string `json:"node2_pub"`
    Capacity float64 `json:"capacity,string"`
}
```

La struttura **NodeData** estrae la chiave pubblica a 256 bit e l'alias del nodo. La struttura **EdgeData** rappresenta il canale di pagamento e salva le chiavi pubbliche dei due partner di canale e la capacità del canale, ovvero la quantità di satoshi bloccata nell'indirizzo multisig 2-di-2. L'utilizzo del tag **,string** per il campo **capacity** è necessario per convertire il campo **capacity** in **float64**, dato che nel file JSON la capacità in satoshi è racchiusa tra virgolette e quindi trattata come stringa.

Per ottimizzare l'utilizzo di memoria RAM, evitando di caricare nella sua completezza il file JSON (tramite la funzione **json.Unmarshal()**), il modulo **parser.go** sfrutta il parsing in streaming. Questo è reso possibile dalla funzione **LoadJSON** utilizzando l'oggetto **Decoder**, inizializzato mediante la funzione **json.NewDecoder()**:

```
func LoadJSON(filename string) ([]NodeData, []EdgeData, error) {
    var nodes []NodeData //slice che contiene i dati dei nodi del grafo JSON
    var edges []EdgeData //slice che contiene i dati degli archi del grafo JSON
    // ... apertura file JSON ...

    decoder := json.NewDecoder(file)
    _, err = decoder.Token() // consuma il carattere '{'
    for decoder.More() {
        t, err := decoder.Token()
        key := t.(string)
        if key == "nodes" {
            decoder.Token() // consuma l'elemento '['
            for decoder.More() {
                var node NodeData
                if err := decoder.Decode(&node); err == nil {
```

```

        nodes = append(nodes, node)
    }
}
decoder.Token() // consuma l'elemento ']'
} else if key == "edges" {
    // ... stesso processo per gli archi
}
}
return nodes, edges, nil
}

```

Il parsing in streaming sfrutta il metodo `decoder.Token()` dell'oggetto `Decoder` per consumare i token e, appena incontra l'array `nodes` o `edges` entra in un ciclo che decodifica e salva all'interno degli slices di tipo `NodeData` o `EdgeData` solamente i tag specificati all'interno delle `struct`.

4.3 Il package analyzer

Una volta eseguito il parsing del file JSON e ottenuti i dati grezzi della rete Lightning, il prossimo passo è quello di convertirli in una struttura dati adeguata per effettuarne l'analisi. Questo processo è reso possibile grazie alle librerie fornite da **Gonum**.

4.3.1 Costruzione del grafo: il file `graph_builder.go`

Il compito della costruzione del grafo a partire dai dati estratti dal file JSON è affidato al file `graph_builder.go`. Come visibile all'interno della documentazione della libreria `gonum/graph` [17] l'oggetto di tipo `Node` utilizza degli identificatori univoci interi a 64 bit (`int64`), mentre nella nostra `struct NodeData` abbiamo mappato le chiavi pubbliche in esadecimale. È necessario definire una struttura che consenta di convertire la chiave pubblica del nodo nell'ID corrispondente e viceversa:

```

type LNGraph struct {
    Graph      *simple.UndirectedGraph
    PubKeyToID map[string]int64
    IDToPubKey map[int64]string
}

```

Questa struttura incapsula il grafo vero e proprio e mantiene due dizionari (`map`) per eseguire l'operazione di conversione in tempo $O(1)$.

Presentiamo ora la `struct` che definisce il canale della rete Lightning. Dato che durante le simulazioni andremo costantemente ad aggiornare il saldo, questa operazione richiederebbe ogni volta di cancellare l'arco vecchio sostituendolo con lo stesso con il saldo modificato, degradando le prestazioni. La soluzione proposta è la seguente:

```

type BalanceState struct {
    BalanceA float64 // bilancio A→B
    BalanceB float64 // bilancio B→A
}

```

```

type LightningEdge struct {
    F, T      graph.Node
    Capacity  float64
    Balance   *BalanceState
}

```

La `struct LightningEdge` implementa l'interfaccia di un arco in Gonum, infatti nel file di simulazione sono state implementate anche tutti suoi metodi come `From()`, `To()` e `ReverseEdge()`. Il campo più importante è il campo `Balance` che, mantenendo un puntatore alla `struct BalanceState`, ha il compito di mantenere lo stato del bilancio tra i due nodi. Questa soluzione permette di separare la struttura della rete dal suo stato economico, infatti, all'avvenire di un pagamento, i valori relativi ai saldi dei due nodi vengono aggiornati accedendo direttamente alla memoria. Questo permette quindi l'aggiornamento in modo immediato, con un tempo $O(1)$.

La funzione `BuildGraph()` ha il compito di prendere in input le strutture dati caricate nella funzione `LoadJSON()` e creare fisicamente in memoria il grafo della rete.

Il primo passo è quella dell'inizializzazione di un nuovo grafo non orientato tramite `simple.NewUndirectedGraph()` e l'allocazione delle due mappe per la conversione da chiave pubblica a ID e viceversa.

```

func BuildGraph(nodi []parser.NodeData, archi []parser.EdgeData) *LNGraph {
    graph := simple.NewUndirectedGraph()
    pubKeyToId := make(map[string]int64)
    idToPubKey := make(map[int64]string)
}

```

In seguito, iterando su tutto lo slice `nodi` di tipo `NodeData`, si popola di nodi il grafo della rete nel seguente modo: viene creato un nodo tramite `graph.NewNode()`, che associa un ID univoco (progressivo) al nodo appena creato e aggiunto al grafo; in seguito vengono associate le coppie (*chiave, valore*) all'interno delle due mappe `pubKeyToId` e `idToPubKey` per la conversione.

```

for _, nData := range nodi {
    node := graph.NewNode()
    graph.AddNode(node)

    pubKeyToId[nData.PubKey] = node.ID()
    idToPubKey[node.ID()] = nData.PubKey
}

```

Dopo aver inserito tutti i vertici, il passaggio successivo è quello di inserire gli archi che andranno a rappresentare i canali di pagamento. Per garantire il corretto inserimento degli archi viene eseguito un doppio controllo. Il primo controllo riguarda l'esistenza dei nodi all'interno della rete e per verificarlo viene sfruttata una sintassi particolare di Go, detta “*comma ok idiom*”.

```
idNode1, existNode1 := pubKeyToId[eData.Node1pub]
idNode2, existNode2 := pubKeyToId[eData.Node2pub]
```

Quando viene interrogata una mappa in Go, il risultato può essere assegnato a due variabili contemporaneamente, come nell'esempio sopra. La prima variabile, ovvero `idNode1`, riceve la conversione dalla chiave pubblica all'ID relativo al nodo richiesto, mentre la seconda variabile, `existNode1`, riceve un valore booleano impostato a `true` se la chiave richiesta è realmente presente all'interno della mappa, altrimenti riceve `false`.

Questo permette quindi di verificare in tempo costante $O(1)$ l'esistenza effettiva dei nodi nella rete. Nel ciclo `for` che itera sulla struttura degli archi vengono effettuati due controlli consecutivi. Il primo `if existNode1 && existNode2` verifica la corretta esistenza dei due nodi, mentre il controllo successivo `if idNode1 != idNode2`, si assicura che l'ID dei due nodi non coincida, evitando di inserire eventuali *self loop* nel grafo.

```
for _, eData := range archi {
    if existNode1 && existNode2 {
        if idNode1 != idNode2 {
            edge := LightningEdge{
                F: graph.Node(idNode1), // From
                T: graph.Node(idNode2), // To
                Capacity: float64(eData.Capacity),
                Balance: &BalanceState{
                    BalanceA: float64(eData.Capacity) / 2.0,
                    BalanceB: float64(eData.Capacity) / 2.0,
                },
            }
            graph.SetEdge(edge)
        }
    }
}

return &LNGraph{
    Graph: graph,
    PubKeyToID: pubKeyToId,
    IDToPubKey: idToPubKey,
}
```

Dopo questi vari controlli viene istanziato l'arco da aggiungere tramite la struttura `LightningEdge`. Osserviamo che, in questo snippet di programma presentato, dato che il bilancio non è comunicato dal *protocollo gossip* per questioni di privacy, per l'analisi del processo randomico abbiamo assunto che inizialmente la capacità dei due partner di canale è equamente divisa.

Dopo aver definito l'arco, questo viene aggiunto all'interno del grafo, tramite il metodo `graph.SetEdge()` e terminiamo la funzione ritornando l'indirizzo di memoria della struttura `LNGraph`, che incapsula il grafo e le mappe di conversione.

La funzione `ErdosRenyiGraph()` sfrutta la funzione `Gnp()` del pacchetto `gonum/graph/graphs/gen`, che accetta come parametro un grafo di destinazione in cui costruire la struttura, il numero n di nodi desiderati e il numero p che indica la probabilità con cui un arco appartiene o meno al grafo. Per associare una capacità in modo imparziale al seguente grafo, i suoi archi sono stati sostituiti con la struttura `LightningEdge` e le capacità sono state scelte in maniera casuale dalle capacità degli archi del grafo della Lightning Network stessa.

4.3.2 Analisi della topologia: il file `metriche.go`

Questo file implementa le varie funzioni per il calcolo delle misure topologiche del grafo, come il grado medio, il numero di componenti connesse o il diametro. Prestiamo particolare attenzione alla funzione per il calcolo delle componenti connesse, che ci permetterà di individuare e calcolare la dimensione della *Giant Component*.

La funzione `ComponentiConnesse()` richiama l'algoritmo `topo.ConnectedComponents()` fornito da `Gonum`, che restituisce una slice bidimensionale di nodi `[][]graph.Node`, in cui ogni slice interna contiene tutti i nodi del grafo che costituiscono una componente connessa.

Iterando su questa slice e verificandone la lunghezza della componente, riusciamo ad individuare quella che connette il maggior numero di nodi, ottenendo così uno slice contenente tutti gli ID dei nodi della Giant Component. In questo modo tramite la funzione `GeneraSottografo()` riusciamo ad estrarne il grafo per analizzarla nel dettaglio.

```
func (lng *LNGraph) ComponentiConnesse() (error, *LNGraph) {
    componentiConnesse := topo.ConnectedComponents(lng.Graph)

    giantComp := []graph.Node{}
    for _, c := range componentiConnesse {
        if len(c) > len(giantComp) {
            giantComp = c
        }
    }
    giantCompGraph := lng.GeneraSottografo(giantComp)
    return err, giantCompGraph
}
```

Seguendo la logica della funzione `BuildGraph()` vista in precedenza, nella seguente funzione osserviamo una piccola variazione nell'inserimento degli archi. Avendo a disposizione solamente uno slice di ID dei nodi, non abbiamo informazioni sugli archi. Iterando su questo slice possiamo ottenere, per ogni nodo v , la lista dei nodi adiacenti grazie al metodo `From()`, per andare in seguito a recuperare l'arco da inserire nel sottografo.

I controlli effettuati per condizionare l'aggiunta dell'arco nel grafo sono dei controlli di esistenza del nodo e osserviamo la sintassi `nodo.ID() < nDest.ID()` che assicura che l'arco esistente tra i due nodi sia prelevato dal metodo `EdgeBetween()` una sola volta, dimezzando le iterazioni per l'inserimento degli archi.

```

func (lng *LNGraph) GeneraSottografo(nodiComp []graph.Node) *LNGraph {
// ... aggiunta dei nodi ...
// stessa logica di BuildGraph()
    for _, nodo := range nodiComp {
        adiacenti := lng.Graph.From(nodo.ID())
        for adiacenti.Next() {
            nDest := adiacenti.Node()
            if sottoGrafo.Node(nDest.ID()) != nil && nodo.ID() < nDest.ID()
{
                eData := lng.Graph.EdgeBetween(nodo.ID(), nDest.ID())
                sottoGrafo.SetEdge(eData)
            }
        }
    }
}

```

4.4 Il package **simulation**

Dopo aver ottenuto il grafo grazie alle funzioni viste nelle sezioni passate, introduciamo il package **simulation** che permette di effettuare gli attacchi mirati e le simulazioni di pagamento su un grafo che modella la Lightning Network.

4.4.1 La scelta dei target: il file **attacking.go**

Nel seguente file viene definito il criterio di scelta dei nodi su cui simulare l'attacco, che nello scenario reale può rappresentare lo spegnimento dei server o un attacco hacker volontario sul nodo. Per individuare i nodi più influenti da attaccare, definiamo la misura di *degree centrality*.

La *degree centrality* è il più semplice indicatore di centralità di un nodo in una rete. La classificazione dei nodi viene eseguita in base al loro grado, ovvero il numero di canali che hanno aperto. In questo modo riusciamo a individuare i *super hub* su cui effettuare gli attacchi mirati con una complessità temporale di $O(|V|)$.

```

func CalcolaDegreeCentrality(lng *analyzer.LNGraph) map[int64]float64 {
    centralityMap := make(map[int64]float64)
    nodi := lng.Graph.Nodes()
    for nodi.Next() {
        node := nodi.Node()
        centralityMap[node.ID()] = float64(lng.Graph.From(node.ID()).Len())
    }
    return centralityMap
}

```

Il funzionamento è semplice: per ogni nodo viene inserita in una map **ID:grado**, la lunghezza dello slice dei nodi adiacenti, che indica proprio il grado del nodo tramite il metodo **Len()** che ha costo costante $O(1)$.

4.4.2 L'implementazione dei pagamenti: il file `payments.go`

Il cuore della simulazione è rappresentato dal file `payments.go`, che contiene la funzione di instradamento di un pagamento, detta `Paga()`. Questa prende come parametri gli ID del nodo sorgente, del nodo di destinazione e la quantità di satoshi da trasferire.

Dato che i saldi sono in continua evoluzione, l'utilizzo di algoritmi predefiniti per il calcolo dello shortest path, come Dijkstra o A* inclusi in `gonum` risultano inefficaci: la validità di un arco è determinata dalla liquidità disponibile in quel preciso istante durante l'esecuzione.

Per risolvere questo problema è stata implementata da zero un'esplorazione basata sulla Breadth-First Search (BFS) [16] con controllo sul vincolo della liquidità.

La funzione utilizza 3 mappe e una coda: `visitati` per evitare i nodi già esplorati, `parentNode` e `parentEdge` per ricostruire il cammino in backtracking dal destinatario e la coda inizializzata col solo nodo sorgente.

```
func Paga(nodoS, nodoD int64, paga float64, lng *analyzer.LNGraph) (bool,
int) {
    visitati := make(map[int64]bool)
    parentNode := make(map[int64]int64)
    parentEdge := make(map[int64]analyzer.LightningEdge)
    coda := []int64{nodoS}
    visitati[nodoS] = true
```

Ad ogni iterazione, viene estratto un nodo v dalla coda secondo la politica FIFO (First-In First-Out) e se questo coincide col nodo destinatario allora la ricerca si interrompe con successo. In caso contrario, si analizzano i nodi adiacenti e si verifica se la liquidità disponibile è sufficiente per far passare il pagamento. Nel caso questa condizione sia verificata, si aggiunge il nodo estratto in coda e si continua la ricerca, altrimenti si prova un nuovo nodo adiacente.

```
    path := false

    for len(coda) > 0 {
        nodo := coda[0]
        coda = coda[1:]

        if nodo == nodoD {
            path = true
            break
        }

        adiacenti := lng.Graph.From(nodo)
        for adiacenti.Next() {
            next := adiacenti.Node().ID()

            if !visitati[next] {
                arco := lng.Graph.EdgeBetween(nodo, next).(LightningEdge)
                if arco.SatoshiDisponibili(nodo) ≥ paga {
```

```

        visitati[next] = true
        parentNode[next] = nodo
        parentEdge[next] = arco
        coda = append(coda, next)
    }
}
}

```

Il punto centrale di questa implementazione è la gestione dei fallimenti. Se la BFS non trova un cammino valido, la funzione sfrutta la funzione `PathExistIn()` del pacchetto `gonum/graph/topo` per individuare se esisteva un percorso tra il nodo sorgente e il destinatario. Grazie a questa procedura possiamo determinare se un pagamento è fallito per mancanza di liquidità o perché non esiste un percorso fisico che collega i due nodi

```

if !path {
    if topo.PathExistsIn(lng.Graph, nodoS, nodoD) {
        return false, 0, true
    } else {
        return false, 0, false
    }
}
}

```

La seconda parte della funzione ha lo scopo di ricostruire il percorso, se trovato, utilizzato per instradare il pagamento. Questo processo è fatto al contrario, partendo dal nodo destinatario del pagamento fino a risalire alla sorgente. Finché non si incontra un nodo uguale alla sorgente del pagamento, si estrae il prossimo nodo dal dizionario dei parent con il rispettivo arco utilizzato e tramite la funzione `AggiornaBilancio()` vengono modificati i bilanci all'interno della struttura `LightningEdge`, in modo costante tramite accesso diretto in memoria grazie alla definizione della struttura `BalanceState`. Infine viene ritornato il valore `true`, che dimostra la corretta riuscita nel pagamento e il numero di hop effettuati durante il percorso.

```

nodo := nodoD
hop := 0

for nodo != nodoS {
    prev := parentNode[nodo]
    channel := parentEdge[nodo]

    channel.AggiornaBilancio(prev, paga)
    nodo = prev
    hop++
}

return true, hop, nil
}

```

4.4.2.1 Il cuore del package: la funzione `calcolaFallimento()`

Questa funzione rappresenta il motore centrale della simulazione, ovvero l'implementazione del processo aleatorio che studiamo in questa tesi.

Il primo step del processo random è l'inizializzazione del generatore casuale di numeri `rand`, utilizzando l'ora esatta al nanosecondo come seed, per garantire che ad ogni esecuzione la sequenza di numeri sia sempre diversa.

```
func calcolaFallimento(lng *analyzer.LNGraph, numP int, valP float64)
(float64, float64, float64) {
    // ... caricamento ID nodi del grafo lng all'interno di un array ...
    rand := rand.New(rand.NewSource(time.Now().UnixNano()))

    pagamentiRiusciti := 0
    failLiquidita := 0
    failPath := 0
    avgHops := 0.0
```

In seguito vengono estratti due indici in modo casuale ed utilizzati per estrarre l'ID del nodo di partenza e il nodo di destinazione dall'array che contiene tutti gli ID dei nodi del grafo.

```
for i := 0; i < numP; i++ {
    startNodeIndex := rand.Intn(numNodi)
    destNodeIndex := rand.Intn(numNodi)
```

Il seguente ciclo `for` (che si comporta come un ciclo `while`) è necessario per assicurarsi che l'estrazione casuale non restituisca lo stesso nodo sia come sorgente che come destinazione del pagamento, prevenendo delle situazioni di pagamento verso sé stessi.

```
for startNodeIndex == destNodeIndex {
    destNodeIndex := rand.Intn(numNodi)
}
startNodeID := IDArray[startNodeIndex]
destNodeID := IDArray[destNodeIndex]
```

Il passaggio successivo è l'esecuzione del pagamento effettuata richiamando la funzione `Paga()` precedentemente presentata avente come nodo di partenza e arrivo gli ID dei due nodi estratti in modo casuale. Sfruttando la capacità di Go nel restituire valori di ritorno multipli, vengono estratti i valori di esito del pagamento, il numero di hop dalla funzione di pagamento e, nel caso in cui il pagamento sia fallito, l'esistenza o meno di un path tra i due nodi. Condizionando su questi valori, incrementiamo i contatori relativi agli esiti.

```
esito, hops, pathExist := Paga(startNodeID, destNodeID, valP, lng)
if esito {
    pagamentiRiusciti++
    totaleHops += hops
```

```
    } else if !esito && !pathExist {  
        failPath++  
    } else {  
        failLiquidità++  
    }  
}  
}
```

Al termine della funzione possiamo utilizzare i contatori sui pagamenti riusciti/falliti per calcolare il tasso di fallimento per mancanza di liquidità, il tasso di fallimento per mancanza di path e la lunghezza media dei percorsi.

Concludiamo dunque la panoramica sull'architettura tecnica del software di simulazione. Grazie alle implementazioni ottimizzate in memoria, il programma è in grado di instradare un grande numero di pagamenti. Nel prossimo capitolo lo strumento di simulazione verrà messo in azione per l'analisi dei dati raccolti per la valutazione della centralità della Lightning Network.

Capitolo 5

Analisi dei risultati

Il seguente capitolo presenta i risultati ottenuti dalle simulazioni condotte sul grafo della Lightning Network e sul grafo $G(n, p)$ generato a partire dal modello di Erdős–Rényi–Gilbert. L’obiettivo è quello di mostrare la dipendenza della Lightning Network dai suoi *super hub* e misurare l’impatto della rimozione di questi.

5.1 La simulazione

Denotiamo con $G_{LN} = (V_{LN}, E_{LN})$ il grafo della Lightning Network, dove $|V_{LN}| = n$ rappresenta il numero di nodi ed $|E_{LN}| = m$ indica il numero di archi, mentre denotiamo con $G_{ER} = (V_{ER}, E_{ER})$ il grafo generato dal modello di Erdős–Rényi, dove $|V_{ER}| = |V_{LN}| = n$ ed $|E_{ER}| \approx |E_{LN}|$.

Per eseguire le simulazioni dei pagamenti tra i due grafi G_{LN} e G_{ER} sono stati utilizzati i seguenti parametri:

- **valorePagamento:** 50.000 satoshi (circa 27 euro al 03/07/2026) per singolo pagamento, per simulare una spesa “normale”.
- **pagamentiPerStep:** 10.000 tentativi di pagamento per ogni step scegliendo u.a.r. sorgente e destinazione.
- **numeroDiStep:** 150, ogni step rappresenta la rimozione di un singolo nodo. Al termine della simulazione vengono dunque rimossi poco meno dell’1% dei nodi totali della rete.

Alla fine di ogni step, la capacità di tutti i canali della rete viene ripristinata al valore iniziale di $\frac{c(e)}{2}$ per ogni $e \in E_{LN} \cup E_{ER}$.

5.2 La Lightning Network

In questa sezione analizzeremo i risultati della simulazione sul grafo della Lightning Network negli scenari di guasti casuali e attacchi mirati, fornendo una spiegazione sulle percentuali di fallimento ottenute.

5.2.1 Impatto dei guasti casuali

Nello scenario dei guasti casuali vengono progressivamente rimossi 150 nodi scelti u.a.r. su tutta la rete. Questo ci permette di analizzare il normale comportamento della rete, in cui degli host possono essere temporaneamente non raggiungibili.

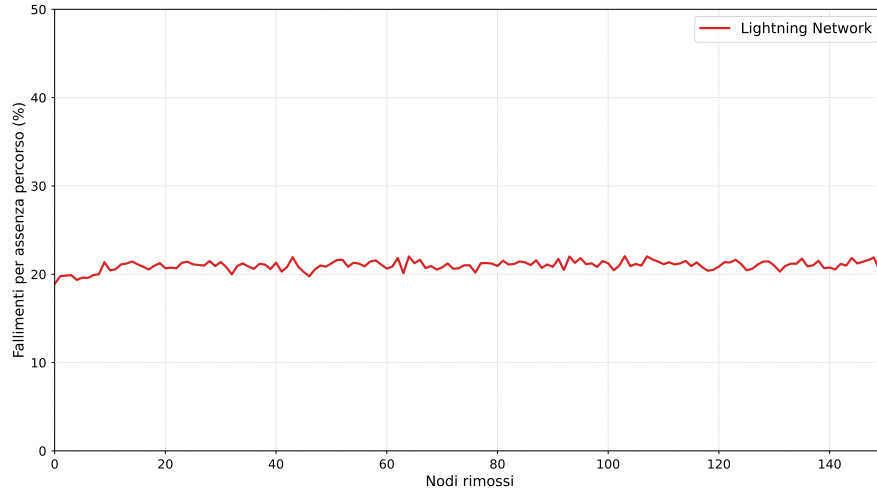


Figura 9: Percentuale di fallimento per mancanza di path (guasti casuali)

Analizziamo la percentuale di fallimento per mancanza di path nel grafo della Lightning Network. Questo valore nasce dall'esecuzione della simulazione su tutto il grafo della Lightning Network, composta di 721 componenti connesse, di cui una *giant component* di 15.547 nodi. Essendo $|V_{LN}| = 17.321$, allora il numero di nodi appartenenti alla giant component sono circa il 90% dei nodi totali. Questo significa che, presi due nodi a caso $u, v \in V_{LN}$, la probabilità $P(E_u \cap E_v)$ dove E_i è l'evento "il nodo i appartiene alla giant component" è

$$P(E_u \cap E_v) = P(E_u) \cdot P(E_v) \approx \frac{9}{10} \cdot \frac{9}{10} = \frac{81}{100} = 0,81$$

Per complementarità, la probabilità che almeno uno dei due nodi non appartenga alla giant component impedendo la connessione topologica è:

$$1 - P(E_u \cap E_v) \approx 0,19$$

Questo significa che circa il 20% dei fallimenti che vediamo in figura avvengono perché sono scelti 2 nodi appartenenti a due componenti connesse distinte.

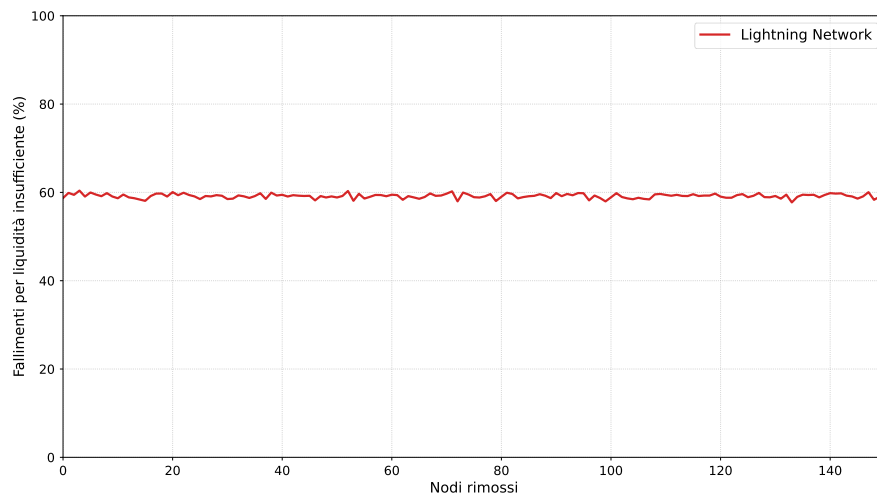


Figura 10: Percentuale di fallimento per assenza di liquidità (guasti casuali)

In [Figura 10] osserviamo come, nel caso i due nodi $u, v \in V_{LN}$ siano nella stessa componente connessa (e con alta probabilità appartengono alla giant component), il tasso di fallimento per mancanza di liquidità si stabilizza attorno al 60%. Questa caratteristica è dovuta alla natura *scale free* della rete. Come osservato in [Sezione 2.2.5], avendo molti nodi con di grado 1, questi sono costretti a passare per percorsi “obbligati”, quindi i successi durante i tentativi di pagamento svuotano progressivamente i canali di questi percorsi, stabilendo un fallimento per liquidità insufficiente intorno al 60%.

Osserviamo che la Lightning Network presenta una buona resistenza al guasto casuale, infatti i valori di fallimento non subiscono delle variazioni importanti, dimostrando che la rimozione casuale di nodi non peggiora la situazione economica e l’efficienza di inoltro dei *super hub*.

5.2.2 Impatto degli attacchi mirati

In questa sezione analizziamo come rimozione dei nodi con la *degree centrality* maggiore degradino criticamente le prestazioni della Lightning Network.

La disconnessione di questi super hub ha conseguenze disastrose sul buon esito dei pagamenti, infatti analizzeremo come varia la percentuale di fallimento per mancanza di percorso all’aumentare dei super hub rimossi ed analizzeremo la variazione del fallimento per mancanza di liquidità.

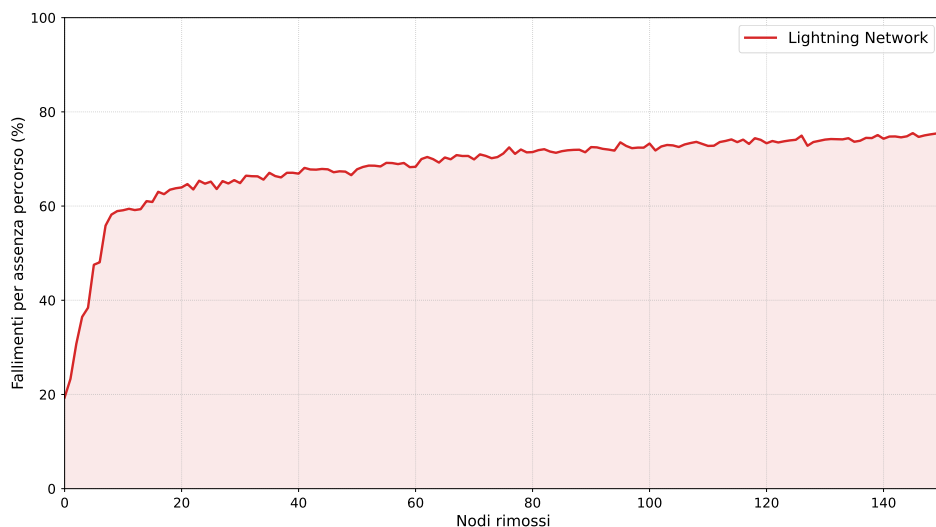


Figura 11: Percentuale di fallimento per mancanza di path (attacchi mirati)

Osserviamo come, coerentemente a quanto visto nella sezione precedente, nel caso in cui il grafo della Lightning Network sia integro, la percentuale di fallimento per mancanza di un percorso tra i due nodi sia del 20%. In seguito rimozione dei primi 5 *super hub* con la *degree centrality* più alta, ovvero col maggior numero di canali aperti nella rete, la percentuale di fallimento è all’incirca raddoppiata. La crescita di questo valore è molto veloce, infatti dopo la sola rimozione dello 0,1% dei *super hub*, la Lightning Network subisce un fallimento a cascata.

Al termine delle 150 rimozioni, la percentuale supera il 75% di fallimento, a causa della frammentazione della rete in migliaia di componenti connesse. Questo ci permette di

osservare come questi grandi hub siano cruciali nel *pathfinding* della rete, dimostrando dunque la loro centralità topologica nel network.

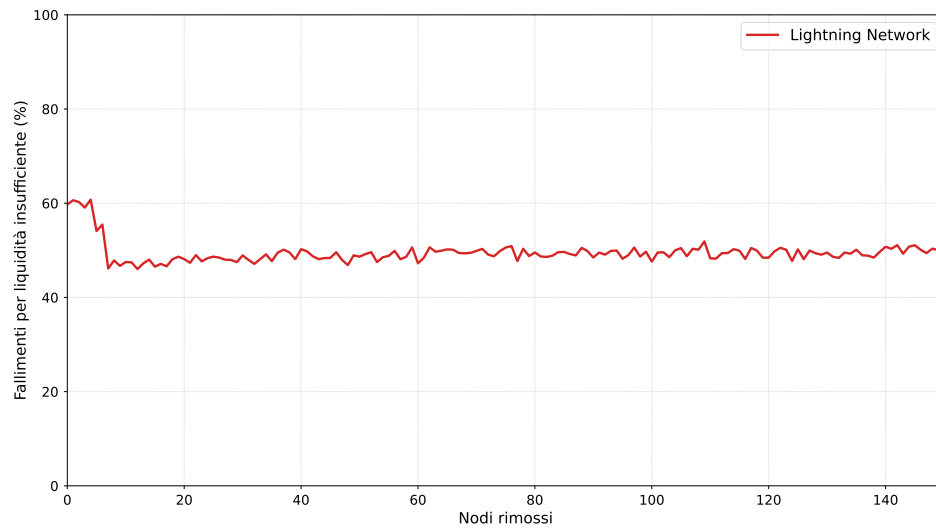


Figura 12: Percentuale di fallimento per assenza di liquidità (attacchi mirati)

In questa figura analizziamo l'impatto degli attacchi mirati sui fallimenti per mancanza di liquidità. È fondamentale osservare che la percentuale di fallimento è stata calcolata escludendo dal campione il numero di pagamenti falliti per assenza di percorso. Questo ci permette di osservare la pura inefficienza economica dei canali sopravvissuti alla disgregazione topologica causata dall'eliminazione dei super hub.

Osserviamo in figura una iniziale flessione della curva dal 60% al 50% in seguito alla rimozione dei primi 5-10 super hub. Questo calo non indica un miglioramento nella capacità di inoltro, ma mostra l'effetto dell'eliminazione dei percorsi che transitano per i nodi centrali. Per i restanti 140 step, la percentuale si stabilizza attorno al 50%, mostrando anche in questo caso come l'esaurimento della liquidità nei canali rimasti influenzi la buona riuscita dei pagamenti.

5.2.3 Aumento del numero di hop

La distruzione della rete per il routing dei pagamenti è confermata anche dall'analisi dell'aumento del numero di salti effettuati nel caso di un pagamento che ha avuto successo. Questa metrica è calcolata come la media degli hop effettuata durante i soli pagamenti andati a buon fine.

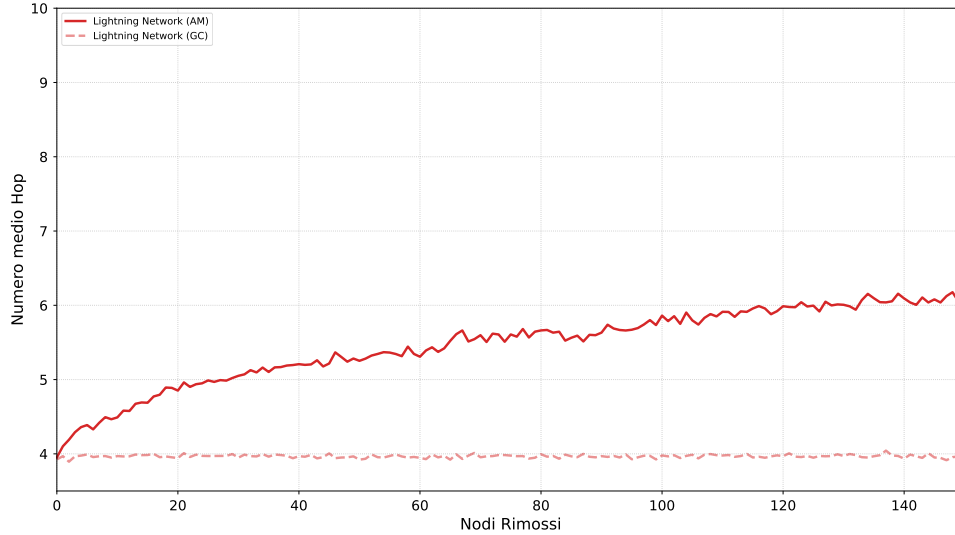


Figura 13: Aumento degli hop medi all’aumentare dei nodi rimossi

In [Figura 13] osserviamo come, gli attacchi mirati sulla Lightning Network (linea continua rossa) costringano l’algoritmo di *pathfinding* ad attraversare percorsi sempre più lunghi all’aumentare della rimozione dei nodi dalla rete.

Questo dato è cruciale perché nel *routing* della Lightning Network la probabilità di successo decade in modo esponenziale all’aumentare della lunghezza del percorso su cui instradare il pagamento, questo perché ogni hop impone il superamento del vincolo di liquidità.

Denotando con $P_c(a) = \frac{c+1-a}{c+1}$ la probabilità di successo del pagamento di una quantità a di satoshi su un singolo canale con capacità c osserviamo come per valori di a che tendono a 0 la probabilità è vicina a 1, mentre per valori di $a > c$ la probabilità scende a 0. Se volessimo ottenere le probabilità di successo su un percorso di più canali, allora la probabilità che il pagamento su quel percorso abbia successo è uguale al prodotto delle probabilità di successo per ogni canale del percorso, quindi

$$P_{\text{pagamento}} = \prod_{i=1}^k P_i$$

dove $P_{\text{pagamento}}$ è la probabilità di successo di un pagamento attraverso k canali e P_i è la singola probabilità di successo sull’ i -esimo canale. Poiché la probabilità di successo su un singolo canale è sempre minore o uguale a 1, allora la probabilità di successo su più canali consecutivi diminuisce in modo esponenziale. [5]

5.3 Il modello di Erdős–Rényi–Gilbert

Per dimostrare che la vulnerabilità della Lightning Network non dipende dalla liquidità dei canali o dal numero di nodi, ma dalla sua struttura topologica, utilizziamo come paragone un modello che ci permette di generare grafi in modo casuale, ovvero il modello di Erdős–Rényi–Gilbert [18].

Per comparare le simulazioni, il grafo del modello di Erdős–Rényi è generato in modo tale da replicare le dimensioni della Lightning Network. In particolare, il valore della

probabilità p è scelto in modo tale da ottenere un numero di archi simili a quelli della LN, quindi abbiamo scelto p tale che:

$$p = \frac{2m}{n(n-1)} = \frac{2 \cdot 50418}{17321^2 - 17321} = \frac{100836}{299999720} = 0.00033612...$$

Generando dunque G_{ER} con parametro p otteniamo che $|E_{ER}| \approx |E_{LN}|$. Successivamente ad ogni arco $e \in E_{ER}$ è stata assegnata una capacità $c(e)$ scelta u.a.r. dalle stesse capacità degli archi del grafo della Lightning Network.

Il modello indicato come $G(n, p)$ genera un grafo $G(V, E)$ di $n = |V|$ nodi, in cui ogni possibile arco e viene inserito con una probabilità fissa p , indipendentemente dagli altri archi. Per questo motivo, la distribuzione di probabilità del grado di ogni vertice $P(k)$ — ovvero la probabilità che, scelto a caso un vertice $v \in V$, questo abbia grado $\deg(v) = k$ — è *binomiale* e per valori elevati di n si approssima ad una *distribuzione di Poisson*. [19]

Graficamente, la distribuzione di probabilità $P(k)$ nel grafo generato dal modello di Erdős–Rényi–Gilbert assume una curva a campana centrata nel grado medio della rete. Questo significa che abbiamo una rete con pochi nodi di grado molto basso o molto alto e tanti nodi con grado vicino al grado medio, quindi abbiamo pochi *super hub*, a differenza della Lightning Network.

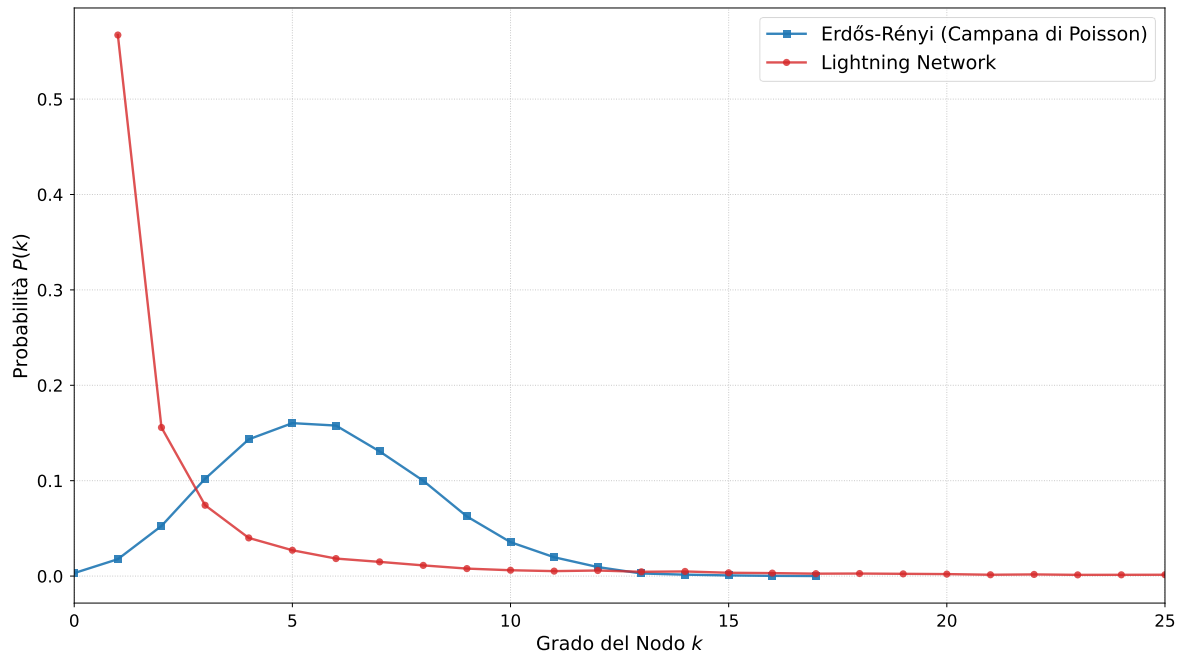


Figura 14: Distribuzione del grado a confronto tra LN ed ER

5.3.1 La distribuzione dei gradi a confronto

Prima di eseguire i pagamenti, analizziamo brevemente la distribuzione dei gradi all'interno dei due grafi. Osserviamo il grafico nella prossima figura.

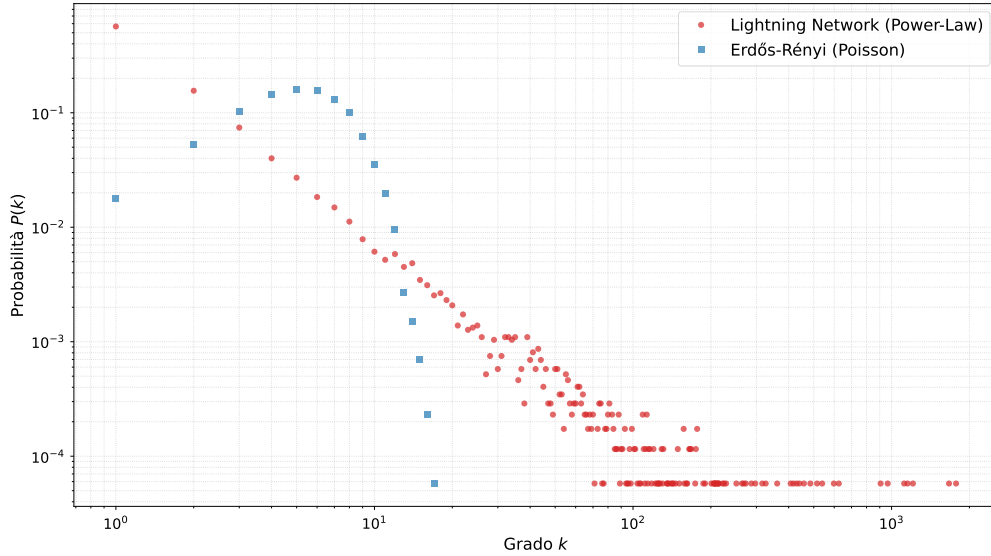


Figura 15: Distribuzione del grado (scala logaritmica)

Possiamo notare come, nel modello di Erdős–Rényi (in blu), la probabilità subisca un crollo dopo aver superato il valore del grado medio (circa 5), confermando l’assenza di hub. In [Figura 14] osserviamo come questo divario è ancora più esplicito, infatti la LN (in rosso) possiede un numero estremamente alto di nodi con grado 1 o 2, in contrasto con la distribuzione a campana che segue il grafo E_{ER} (in blu).

5.3.2 Impatto dei guasti casuali

Come per la Lightning Network, simuliamo lo spegnimento di 150 nodi scelti casualmente e analizziamo come questo influenza l’esito dei pagamenti sul grafo generato dal modello di Erdős–Rényi.

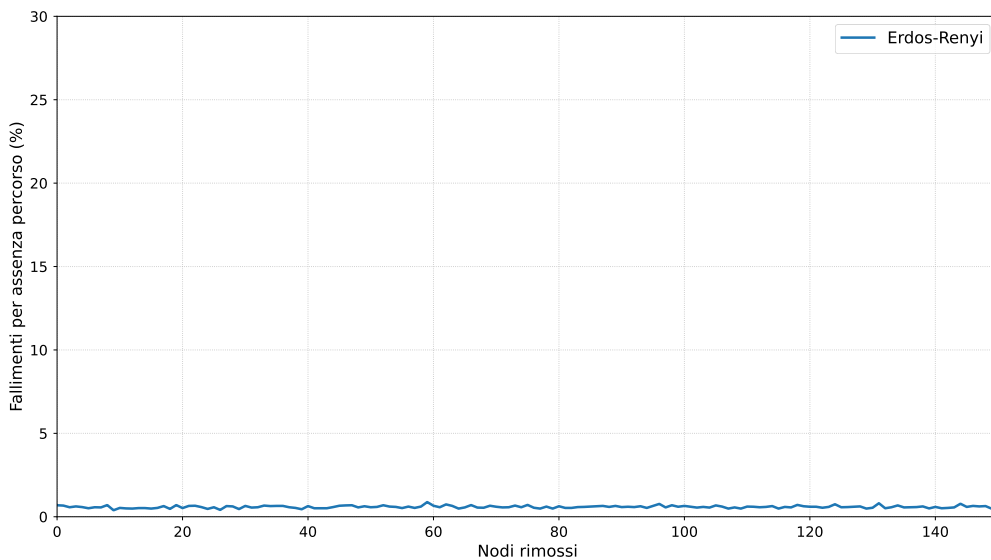


Figura 16: Percentuale di fallimento per mancanza di path (guasti casuali)

Nel grafo G_{ER} , il cui numero di nodi n è identico a quello della Lightning Network e abbiamo un numero molto simile di archi, il numero di componenti connesse è molto basso (circa 55) e abbiamo una singola *giant component*, dato che $np = c > 1$, come

mostrato da Erdős e Rényi in [20]. Questa componente contiene oltre il 99% degli archi, motivo per cui il fallimento per mancanza di path nella simulazione si attesta attorno all'1%.

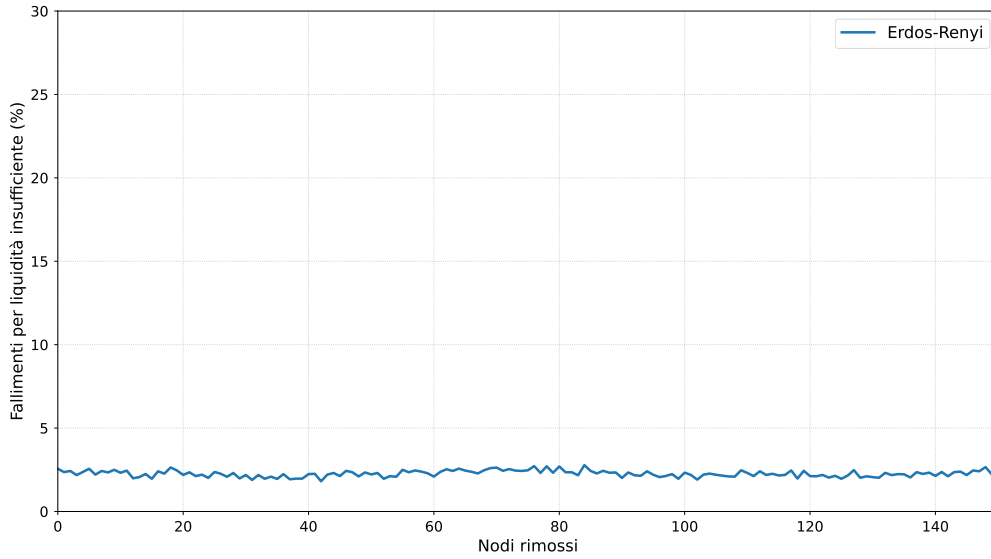


Figura 17: Percentuale di fallimento per liquidità (guasti casuali)

Dato che nel grafo G_{ER} abbiamo osservato che non sono presenti *super hub* con un grado molto maggiore rispetto tutti gli altri nodi, ma che la maggior parte dei nodi hanno circa lo stesso grado, allora significa che presi u.a.r. due nodi $u, v \in V_{ER}$, se questi fanno parte della stessa componente, allora tra loro ci sono molte più percorsi alternativi su cui instradare il pagamento, bilanciando il carico dei pagamenti su più canali, senza mai esaurire un singolo canale. Questo si traduce in un fallimento per mancanza di liquidità che si attesta intorno al 2%.

5.3.3 Impatto degli attacchi mirati

In questa sezione mostriamo come la rete costruita dal modello di Erdős-Rényi confermi la sua resistenza anche in presenza di attacchi mirati sui suoi nodi a centralità maggiore.

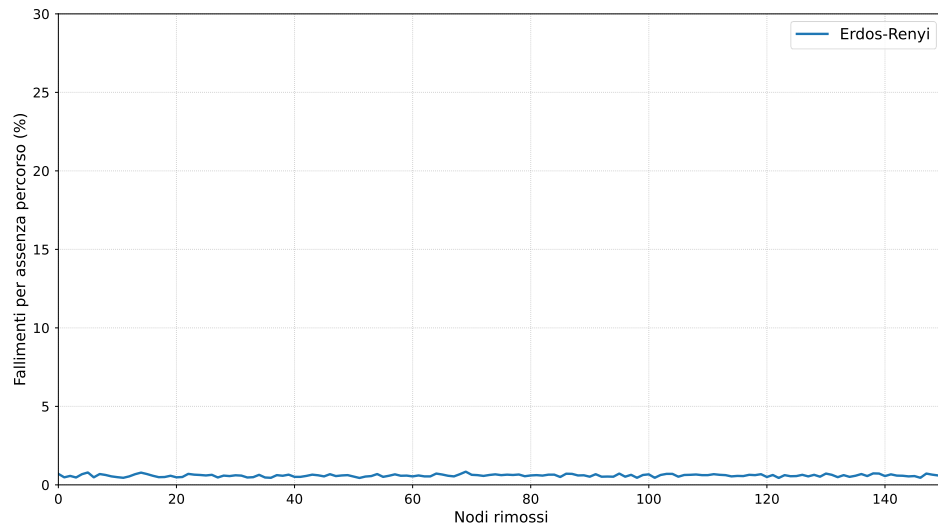


Figura 18: Percentuale di fallimento per mancanza di path (attacchi mirati)

È interessante osservare come si comporta la percentuale di fallimento per mancanza di path durante la simulazione degli attacchi mirati. Notiamo come, all'aumentare dei nodi rimossi, questa percentuale fluttua attorno gli stessi valori, a conferma di quanto il grafo G_{ER} sia privo di grandi nodi la cui rimozione influenzi profondamente la topologia della rete, al contrario di quanto osservato con la Lightning Network. Osservando il grafico, possiamo affermare che la rete assorbe la rimozione dei suoi nodi più centrali senza gravi conseguenze, mantenendo la percentuale di fallimento per assenza di path stabile attorno l'1%.

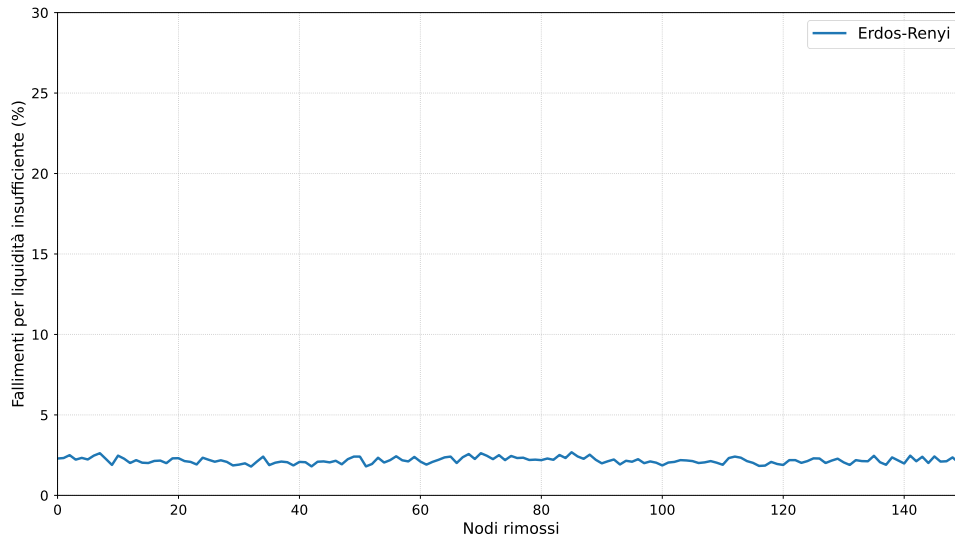


Figura 19: Percentuale di fallimento per assenza di liquidità (attacchi mirati)

Essendo la rete resistente agli attacchi mirati, tende a disgregarsi molto di meno, per questo la percentuale di fallimento per mancanza di liquidità tende a comportarsi allo stesso modo dello scenario di guasti casuali discusso nella [Sezione 5.3.2] precedente.

5.3.4 Aumento del numero di hop

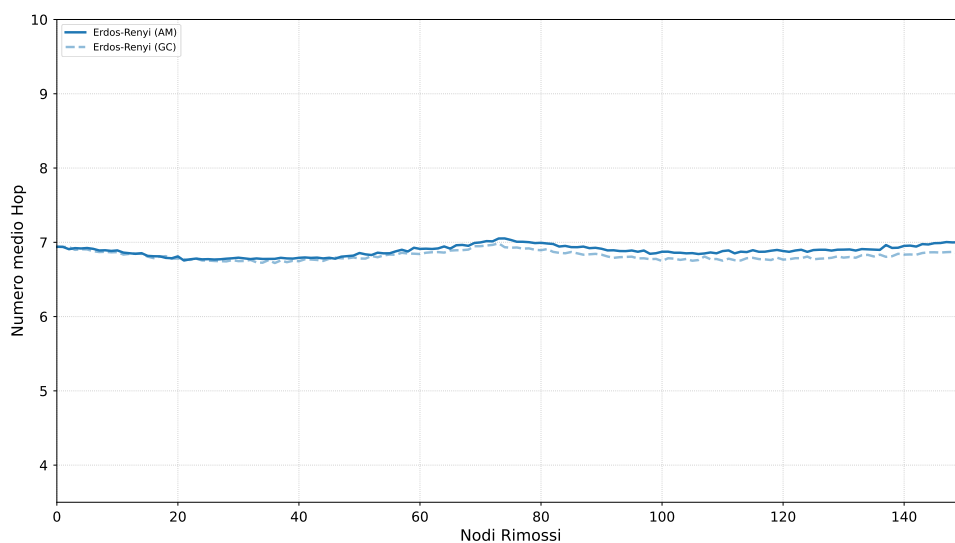


Figura 20: Aumento degli hop medi all'aumentare dei nodi rimossi

Come possiamo osservare, l'andamento del numero medio di hop nel grafo di Erdős-Rényi presenta grandi differenze rispetto quanto osservato nella Lightning Network. Sia nello scenario di guasti casuali (linea tratteggiata) che in quello degli attacchi mirati (linea continua), la lunghezza media dei percorsi si mantiene pressoché costante attorno a un valore di poco inferiore a 7 hop, senza mostrare alcuna tendenza alla crescita all'aumentare delle rimozioni.

Questa stabilità è giustificata dalla omogeneità del grafo G_{ER} : abbiamo osservato dalla distribuzioni dei gradi, che non esistono super hub in grado di fungere da scorciatoie nei pagamenti. Di conseguenza, avendo ogni nodo all'incirca lo stesso grado, l'algoritmo di pathfinding dispone di più rotte alternative su cui instradare il pagamento.

5.4 Confronto finale

In questa breve sezione vengono mostrati i grafici di fallimento sovrapposti nei 2 scenari sulle reti appena presentate, per enfatizzare la differenza strutturale dei due network.

5.4.1 Guasti casuali

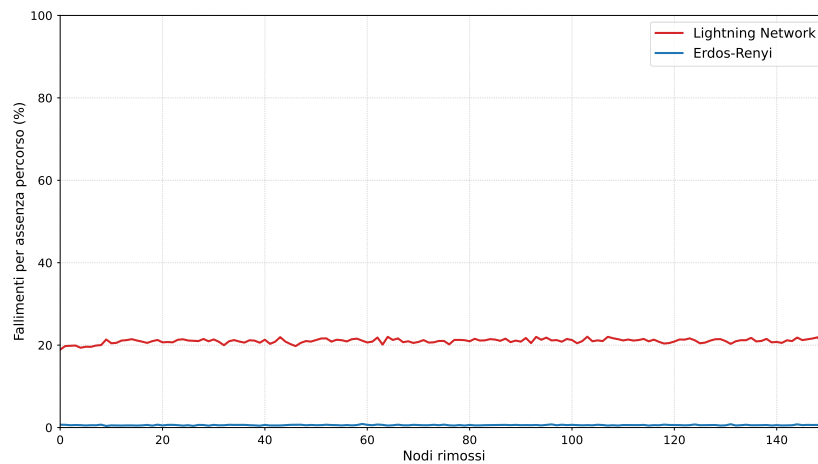


Figura 21: Fallimento mancanza path a confronto (guasti casuali)

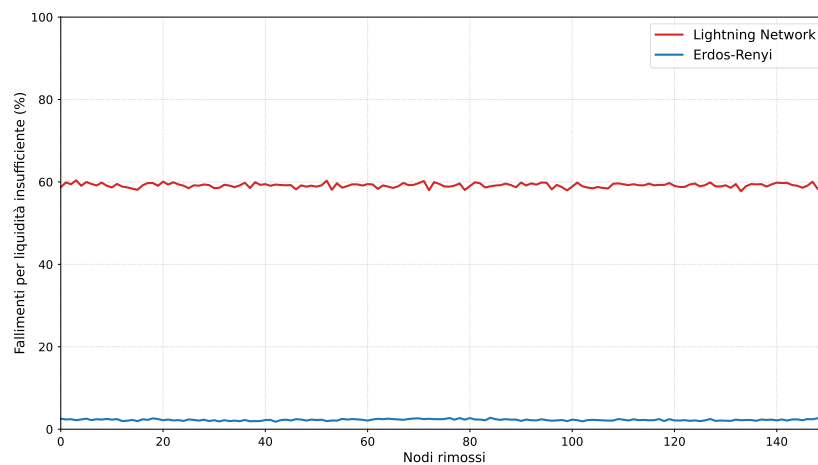


Figura 22: Fallimento liquidità a confronto (guasti casuali)

5.4.2 Attacchi mirati

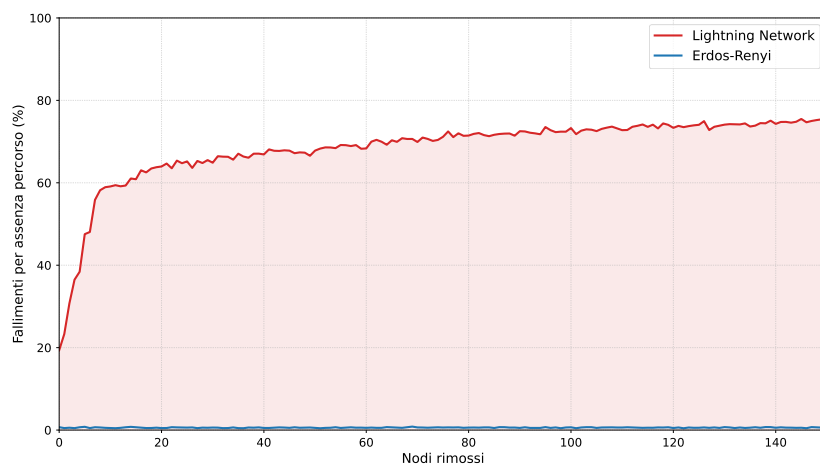


Figura 23: Fallimento mancanza path a confronto (attacchi mirati)

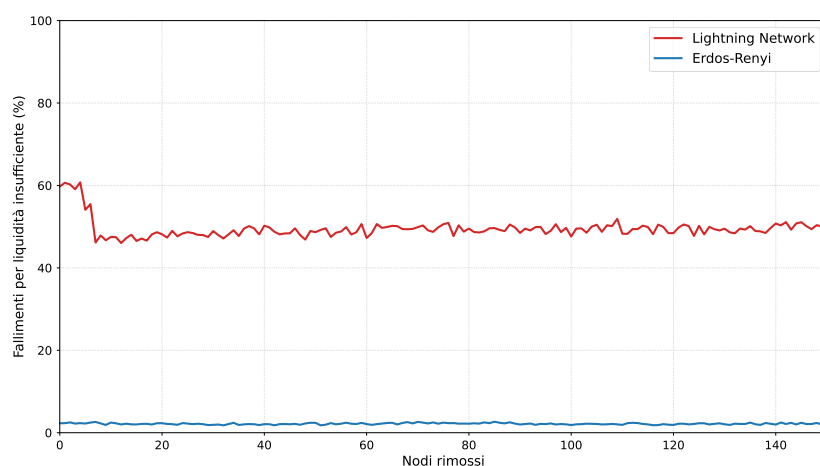


Figura 24: Fallimento liquidità a confronto (attacchi mirati)

5.4.3 Aumento hop

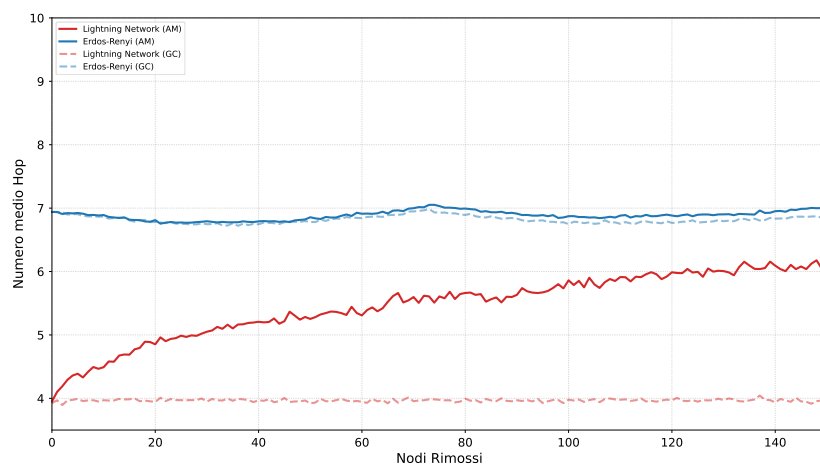


Figura 25: Aumento hop a confronto

Capitolo 6

Sviluppi futuri

Il seguente lavoro ha gettato le basi per l'implementazioni di un software dedicato alla simulazione e all'analisi della Lightning Network, permettendo di quantificare le sue fragilità. Data la natura dinamica e molto più complessa del protocollo, il simulatore può essere esteso per l'analisi di ulteriori metriche e per la simulazione in diversi scenari.

Nella presente simulazione, i pagamenti sono eseguiti seguendo un singolo percorso con un pagamento unico (*Single-Path Payment*). Nella realtà, la Lightning Network ha introdotto la funzionalità degli *Atomic Multipart Payments*, che consente di suddividere un pagamento — che non può essere instradato su un singolo canale — in parti più piccole, instradate verso path differenti. La suddivisione del pagamento in più quote di dimensione inferiore aumenterebbe la probabilità di riuscita, poiché i pagamenti più piccoli hanno maggiore probabilità di successo, in base a quanto visto in [Sezione 5.3.4]. L'introduzione di questa funzionalità consentirebbe di testare se questa tecnica possa effettivamente aumentare il tasso di successo dei pagamenti, limitando i fallimenti per mancanza di liquidità.

Un ulteriore sviluppo può riguardare l'integrazione di algoritmi di *pathfinding* avanzati, in alternativa alla BFS utilizzata nel software. Sarà possibile implementare diverse strategie di instradamento ed eseguire le simulazioni comparative per misurare la variazione del tasso di successo in base alla strategia di ricerca del percorso adottata.

Infine, è possibile estendere l'analisi introducendo il calcolo delle commissioni (*fee*) per i nodi che fungono da intermediari nel percorso. Questo permetterebbe di analizzare, negli scenari di guasti casuali e attacchi mirati già affrontati in questa tesi, come variano i costi totali per l'utente, individuando le migliori strategie di routing per minimizzare questi costi.

Capitolo 7

Conclusioni

La seguente tesi ha affrontato la tenuta strutturale e la stabilità operativa della Lightning Network, con l'obiettivo di verificare come la centralizzazione topologica, dovuta all'ascesa dei super hub, influenzi l'affidabilità del layer 2 di Bitcoin.

Attraverso lo sviluppo di un simulatore di pagamenti in Go, è stato possibile modellare il network come un grafo e simulare il comportamento della rete sotto condizioni di stress, confrontando i guasti casuali con gli attacchi mirati dei nodi con la centralità maggiore. Al termine delle simulazioni possiamo trarre le seguenti conclusioni.

In primo luogo le analisi hanno confermato che la topologia della Lightning Network segue una distribuzione tipica delle reti *scale-free* [14], detta *legge di potenza*. Questa caratteristica garantisce un'efficienza ottimale nell'instradamento dei pagamenti in condizioni normali, ma espone il sistema alla dipendenza da un numero ristretto di nodi centrali. La simulazione ha dimostrato che, mentre la rete dimostra una robustezza nel caso di guasti casuali, essa risulta particolarmente vulnerabile nel caso di attacchi mirati. Abbiamo dimostrato che, la progressiva rimozione dell'1% dei nodi più importanti della rete ha causato una serie di fallimenti a cascata, portando ad una frammentazione della rete in più componenti connesse e degradando la capacità di instradare pagamenti tra gli utenti.

In secondo luogo, l'analisi ha fatto emergere anche la fragilità economica della Lightning Network. Calcolando la percentuale di fallimenti per liquidità, la tesi ha dimostrato che la rete periferica ai super hub è incapace di sostenere i pagamenti di media entità. I super hub non operano unicamente da ponti per il pathfinding, ma rappresentano degli insostituibili centri di liquidità. La loro rimozione obbliga le poche rotte rimaste ad essere utilizzate e le condanna al prosciugamento della liquidità in quei canali.

Infine l'ultimo obiettivo raggiunto è stato quello di osservare che, se da un lato la centralizzazione in pochi grandi nodi si sia rivelata necessaria per risolvere il problema della scalabilità del layer 1 in Bitcoin, dall'altro lato questa configurazione cade in contraddizione con i principi di decentralizzazione che il protocollo Bitcoin ha voluto raggiungere. L'analisi dimostra che l'efficienza ottenuta grazie ai super hub rende la rete più fragile di fronti ad attacchi malevoli. In questo caso possiamo confermare quanto viene affermato nel *Blockchain Trilemma* [3], ovvero che la Lightning Network, per massimizzare la scalabilità delle transazioni e preservare la sicurezza grazie alla crittografia, ha dovuto sacrificare la caratteristica di decentralizzazione.

Bibliografia

- [1] Satoshi Nakamoto, «Bitcoin: A peer-to-peer electronic cash system», 2008.
- [2] E. A. Brewer, «Towards robust distributed systems», 2000.
- [3] T. Nakai, A. Sakurai, S. Hironaka, e K. Shudo, «A formulation of the trilemma in proof of work blockchain», *IEEE Access*, vol. 12, pp. 80559–80578, 2024.
- [4] J. Poon e T. Dryja, «The bitcoin lightning network: scalable off-chain instant payments», 2015.
- [5] A. M. Antonopoulos, *Mastering Bitcoin: unlocking digital cryptocurrencies*. " O'Reilly Media, Inc.", 2014.
- [6] W. Diffie e M. E. Hellman, «New directions in cryptography», in *Democratizing cryptography: the work of Whitfield Diffie and Martin Hellman*, 2022, pp. 365–390.
- [7] N. Koblitz, «Elliptic curve cryptosystems», *Mathematics of computation*, vol. 48, fasc. 177, pp. 203–209, 1987.
- [8] Certicom Research, «SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0». <https://www.secg.org/sec2-v2.pdf>, 2010.
- [9] D. Johnson, A. Menezes, e S. Vanstone, «The elliptic curve digital signature algorithm (ECDSA)», *International journal of information security*, vol. 1, fasc. 1, pp. 36–63, 2001.
- [10] H. Gilbert e H. Handschuh, «Security analysis of SHA-256 and sisters», in *International workshop on selected areas in cryptography*, 2003, pp. 175–193.
- [11] D. Goldschlag, M. Reed, e P. Syverson, «Onion routing», *Communications of the ACM*, vol. 42, fasc. 2, pp. 39–41, 1999.
- [12] G. Danezis e I. Goldberg, «Sphinx: A compact and provably secure mix format», in *2009 30th IEEE Symposium on Security and Privacy*, 2009, pp. 269–282.
- [13] K. Birman, «The promise, and limitations, of gossip protocols», *ACM SIGOPS Operating Systems Review*, vol. 41, fasc. 5, pp. 8–13, 2007.
- [14] A.-L. Barabási e E. Bonabeau, «Scale-free networks», *Scientific american*, vol. 288, fasc. 5, pp. 60–69, 2003.
- [15] T. E. Abedesselam, F. Giacomelli, F. Pasquale, e M. Salvi, «Payment-failure times for random Lightning paths», in *2025 7th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, 2025, pp. 1–10.
- [16] D. C. Kozen, «Depth-first and breadth-first search», in *The design and analysis of algorithms*, Springer, 1992, pp. 19–24.
- [17] Gonum Authors, «Gonum Graph API Documentation». <https://pkg.go.dev/gonum.org/v1/gonum/graph>, 2026.

-
- [18] P. ERDdS e A. R&wi, «On random graphs I», *Publ. math. debrecen*, vol. 6, fasc. 290–297, p. 18, 1959.
 - [19] M. E. Newman, S. H. Strogatz, e D. J. Watts, «Random graphs with arbitrary degree distributions and their applications», *Physical review E*, vol. 64, fasc. 2, p. 26118, 2001.
 - [20] P. Erdős, A. Rényi, e others, «On the evolution of random graphs», *Publications of the*, 1960.